CSE 816 — Software Production Engineering

Final Project Report

FMP-68

A DevSecOps SDLC Automation Platform

with selective-rebuild CI/CD, container image scanning,
in-cluster secrets vaulting, and dashboards-as-code observability

---

Built around a realistic NestJS microservices workload:
Google-OAuth identity, online presence, and live GPS path-tracking.

Author

Shikhar Mutta

Source Code Repository

`github.com/shikhar-mutta/FMP-68`

Submitted in partial fulfilment of CSE 816 — Software Production Engineering
Final Project, Spring 2026

Contents

List of Tables

1 System Overview

FMP-68 is a full-stack microservices platform whose primary contribution is not the application code itself but the *DevSecOps software-development-lifecycle (SDLC) automation* built around it. The reference application — a Google-OAuth identity service with real-time user presence and live GPS path-tracking — is intentionally non-trivial: it spans six independently deployable backend services, a React single-page application, WebSocket fan-out, and a message broker, so that the DevSecOps tooling has a realistic workload to exercise end-to-end.

The system is organised into three concentric layers, each handled by a distinct set of tools and disciplines:

- **Application Layer.** A React 18 single-page app served by Nginx, talking to six NestJS 11 microservices behind an API gateway, persisting through Prisma 5 against a MongoDB replica set, and broadcasting live location updates over Socket.io.
- **Platform Layer.** All workloads run inside a Kubernetes (Minikube) cluster, with Docker as the packaging format, an NGINX ingress controller as the only externally exposed endpoint, RabbitMQ and Redis as supporting infrastructure, and HashiCorp Vault as the in-cluster secrets store.
- **DevSecOps Layer.** Eight Jenkins pipelines (one root orchestrator plus seven service-level pipelines) drive a selective-rebuild CI/CD flow that includes a container image scanning stage, a Docker Hub publish stage, and a zero-downtime rollout step. Ansible handles cluster provisioning through four modular roles, and an ELK stack with auto-imported Kibana dashboards provides the audit trail.

1.1 Project Identity and Rubric Framing

Per the CSE 816 final-project rubric, the *domain* of this work is **DevSecOps**, not generic full-stack web development. Every control listed in the rubric — version control, CI/CD, containerisation, configuration management, orchestration, monitoring/logging, secrets management, autoscaling, and innovation — is implemented end-to-end and demonstrable in a single `git push`. The application is a vehicle for those controls; if the application were replaced with any other multi-service workload, the platform would be re-usable wholesale.

1.2 User Roles in the Reference Application

- **End User.** Signs in with Google, sees who else is online, publishes a path (a series of geo-tagged waypoints), and follows other users' paths in real time on a map.
- **Path Publisher.** Same as an end user, but specifically the originator of a path. Initiates live tracking and broadcasts their position over WebSocket.
- **Path Follower.** Subscribes to a publisher's live broadcast, sees publisher position on a shared map, and may contribute their own position so the publisher knows who is following along.
- **Platform Operator.** Pushes code, watches Jenkins, approves a CVE finding, queries Kibana, scales a deployment. This is the *primary* role the project is designed around.

1.3 Core Functional Capabilities

- Google OAuth 2.0 sign-in with JWT-based session tokens.
- Real-time online/offline presence indicator per user.

- Path publishing, follow requests, follower management.
- Live GPS streaming over Socket.io rooms, with pause / resume / end controls and post-session replay.
- Asynchronous notification fan-out via RabbitMQ (consumer-service scaffolded, wiring stubbed).

1.4 Core DevSecOps Capabilities

- Selective-rebuild Jenkins CI/CD with parallel per-service jobs.
- Multi-stage non-root Docker builds for every service.
- Kubernetes `RollingUpdate` with `maxUnavailable=0` for zero-downtime patching.
- Horizontal Pod Autoscaler (HPA) on the latency-sensitive api-gateway and tracking-service.
- In-cluster HashiCorp Vault for secrets, auto-seeded on pod start via a Kubernetes `postStart` lifecycle hook.
- ELK stack with a pre-built Kibana dashboard checked into Git and re-applied on every fresh cluster via a Kubernetes Job.
- Ansible playbook with four modular roles (`common`, `minikube`, `vault`, `deploy-fmp`) for repeatable cluster provisioning.

1.5 High-Level Request Flow

Figure 1 traces a representative user action (*follow a published path and join its live broadcast*) through every layer of the system.

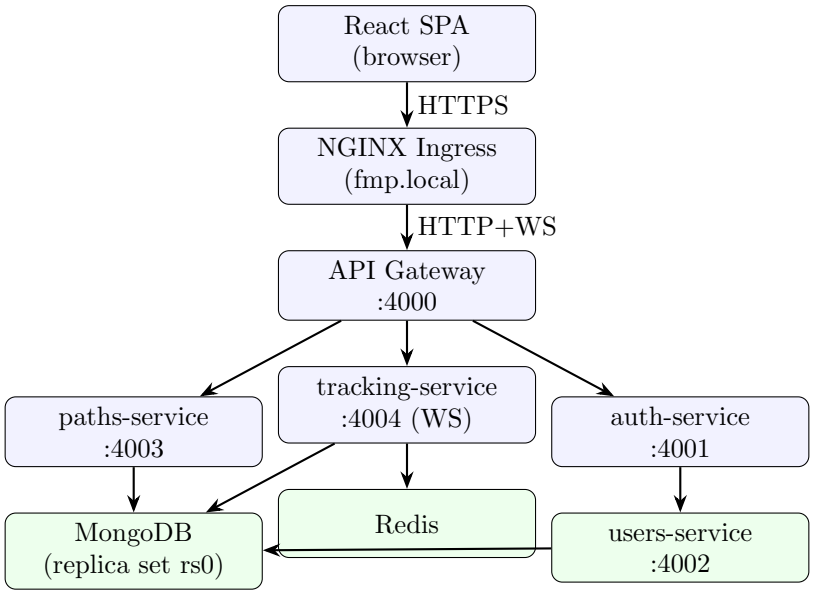


Figure 1: High-level request flow: a follower action traverses the ingress, gateway, paths-service (REST), and tracking-service (WebSocket), with shared MongoDB persistence.

1.6 Why DevSecOps, Not "Generic Web App"

A traditional full-stack project is graded on application features. This project deliberately inverts that hierarchy: the application is the workload, and the graded artefact is the platform. Concretely, every commit to the repository must traverse *five* automated gates before reaching

the cluster: a path-scoped change detection, a unit-test stage, a multi-stage Docker build, a container image scan, and a Kubernetes rollout watch. Each of those gates is implemented in configuration that lives in the repository, so the platform itself is versioned, reviewable, and reproducible.

2 Overall System Architecture

2.1 Architectural Style

The system follows a cloud-native microservices architecture in which each business capability is a separately deployable NestJS application with its own container image, its own Jenkins pipeline, its own Kubernetes Deployment object, and its own resource budget. Three properties drive every architectural decision:

- **Loose coupling at the wire level.** Services communicate over HTTP and RabbitMQ; there is no shared in-process state and no shared compiled library.
- **Single ingress, gateway as policy point.** The cluster has exactly one externally reachable HTTP endpoint (the NGINX ingress controller); everything else is private to the Kubernetes pod network. The API gateway is the only place where edge-level JWT verification happens.
- **Stateful infrastructure runs in known places.** MongoDB lives outside the cluster (developer host), RabbitMQ and Redis live inside; secrets live in Vault. This gives a clear failure domain story per stateful component.

2.2 Logical Architecture Diagram

Figure 2 shows every running container and the external dependencies in one diagram.

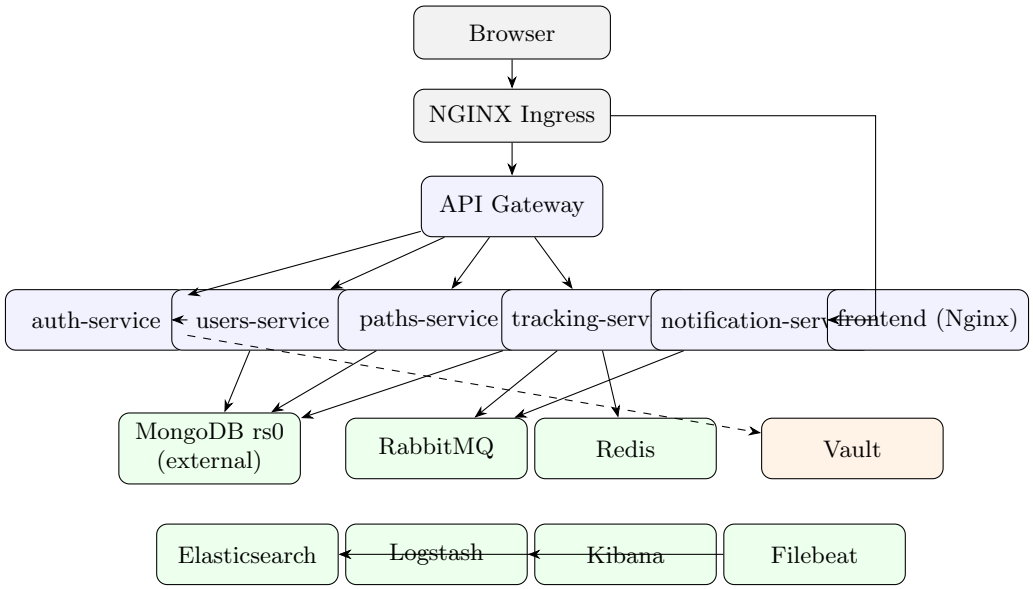


Figure 2: Logical architecture. Blue = NestJS microservices, green = infrastructure, orange = security/secrets, dashed = secrets retrieval at boot.

2.3 Frontend Architecture

The web tier is a React 18 single-page application served by an Nginx container. It speaks REST over HTTP to the API gateway and Socket.io over the same gateway endpoint for live

tracking; it never addresses any backend microservice directly. Sessions are JWT-based, with the token kept in `localStorage` under the key `fmp68_token` and auto-expired after 30 minutes of client-side inactivity.

2.4 Backend Architecture

The backend is six NestJS 11 services, each compiled to JavaScript at container build time and started by `node dist/main`. Every service exposes a `/health` endpoint that the Docker Compose healthchecks and Kubernetes readiness probes consume. The services are listed in Table 1.

Service	Port	State	Responsibility
api-gateway	4000	stateless	Edge proxy, JWT verification, WebSocket upgrade.
auth-service	4001	stateless	Google OAuth 2.0, JWT issuance.
users-service	4002	DB-backed	User CRUD, online/offline tracking.
paths-service	4003	DB-backed	Paths, followers, follow-requests.
tracking-service	4004	DB-backed + WS	Socket.io rooms, GPS persistence.
notification-service	4005	async (scaffolded)	RabbitMQ consumer, intended email fan-out.

Table 1: Backend microservice inventory. All services share a single MongoDB cluster but each has its own Prisma schema namespace.

2.5 Database Architecture

All services share a single MongoDB replica set (`rs0`) but each service maintains its *own* Prisma schema and connects through its *own* `DATABASE_URL` environment variable. In production this is straightforward to split into per-service clusters because no service imports another service's Prisma client; the boundary is enforced at the language level.

2.6 Inter-Service Communication

Two patterns are in active use:

- **Synchronous REST.** auth-service calls users-service over plain HTTP at `http://users-service:4002`, using the `USERS_SERVICE_URL` environment variable.
- **Asynchronous AMQP (scaffolded).** RabbitMQ is deployed and other services can publish to it; the consumer side (notification-service) is scaffolded but not wired to a downstream email transport.

A third pattern, **WebSocket fan-out**, is unique to tracking-service: a single Socket.io server holds rooms named `path-{pathId}` into which the publisher and all followers join, and broadcasts location updates to every member of the room.

2.7 Service Discovery and Dynamic Routing

There is no Eureka or Consul. Service discovery is handled entirely by Kubernetes' built-in DNS: every Service object publishes a stable record at `<svc>.<namespace>.svc.cluster.local`. Inside the `fmp` namespace, a short form (`auth-service:4001`, `users-service:4002`, `paths-service:4003`, ...) is sufficient. The API gateway resolves these names at request time; because Kubernetes Services are virtual IPs backed by kube-proxy, the gateway transparently load-balances across all healthy pod replicas of the downstream service.

2.8 Deployment Architecture

The cluster is partitioned into three namespaces:

- `fmp` — all application microservices, RabbitMQ, Redis, ingress rules.
- `elk` — Elasticsearch, Logstash, Kibana, Filebeat.
- `vault` — HashiCorp Vault (dev mode).

This separation keeps the application's resource quotas independent of the observability and secrets infrastructure, and allows namespace-scoped RBAC to be tightened independently per layer.

2.9 Observability Architecture

Every container's stdout/stderr is collected by a Filebeat DaemonSet, forwarded to Logstash for enrichment (most notably promoting `kubernetes.container.name` into a top-level `service` field), indexed into Elasticsearch under the daily pattern `fmp-logs-YYYY.MM.dd`, and visualised through a pre-built Kibana dashboard that is re-imported on every fresh cluster boot.

2.10 Security Architecture

Security is enforced at four layers:

1. **Identity.** Google OAuth 2.0 for human users, JWT for service requests, with the gateway as the single enforcement point.
2. **Secrets.** HashiCorp Vault in-cluster, with secrets seeded at pod `postStart` time rather than stored in Kubernetes Secrets directly.
3. **Supply chain.** Multi-stage non-root Docker images; Trivy CVE scan stage architected into every per-service pipeline (currently educational, designed to be flipped to enforcing).
4. **Runtime.** Network isolation via Kubernetes namespaces, least-privilege Filebeat ServiceAccount scoped through a dedicated ClusterRole.

3 Technology Stack

The technology stack of FMP-68 spans nine functional layers and is chosen deliberately for the DevSecOps story rather than for novelty in any one slot. Figure 3 summarises the stack at a glance.

3.1 Frontend Technologies

The frontend is React 18 with functional components and hooks. Routing is React Router 6. Maps use React Leaflet 4 over OpenStreetMap tiles. Live tracking uses Socket.io-client 4.8 against the `/tracking` namespace exposed by tracking-service. Toast notifications, theme switching, and authentication context are implemented as separate React Context providers stacked at the application root.

3.2 Backend Technologies

All six backend services use NestJS 11 with TypeScript. Authentication uses `passport-google-oauth20` and `@nestjs/jwt`. HTTP proxying at the gateway uses `http-proxy-middleware` mounted as Express middleware on top of the NestJS bootstrap, which gives transparent WebSocket-upgrade proxying for free. Persistence is through Prisma 5 with the MongoDB provider.

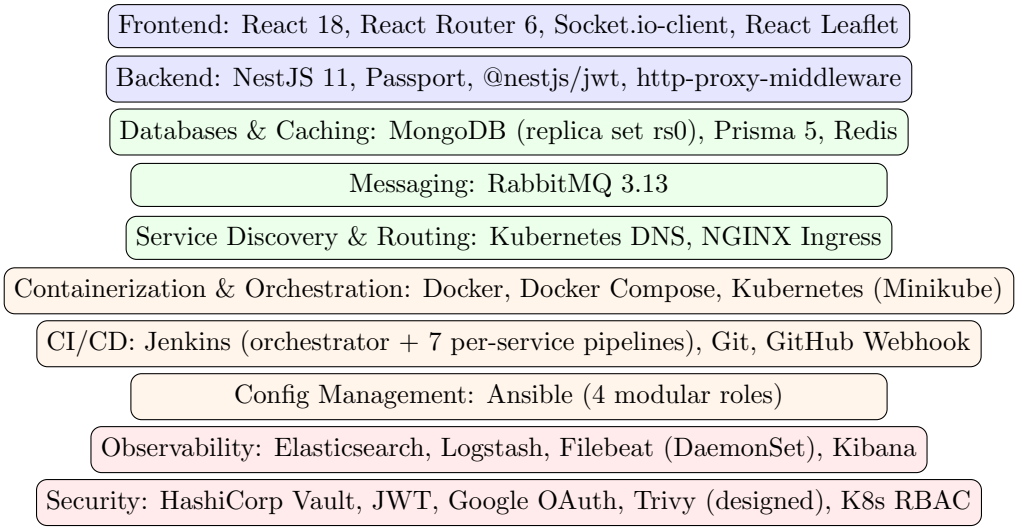


Figure 3: Layered technology stack.

3.3 Databases and Caching

MongoDB is the system of record for all persistent data: users, paths, follow requests, tracking sessions. It runs as a single-member replica set (`rs0`) on the developer host and is consumed in-cluster through a `Service` of type `ExternalName` that points to `host.docker.internal:27017`. Redis is deployed inside the cluster and used for short-lived caching by tracking-service. Prisma 5 is the ORM; each service ships its own `schema.prisma` so schemas evolve independently.

3.4 Messaging and Event Processing

RabbitMQ 3.13 (management variant) runs in-cluster on the AMQP port 5672 and the management UI on 15672. The intended use is asynchronous notification fan-out — when a follow-request is approved or a tracking session ends, an event would be published and notification-service would consume it to send the email. The broker, exchange, and queue plumbing is deployed; the consumer is scaffolded but not yet bound to an outbound email transport.

3.5 Service Discovery and Routing

Kubernetes provides DNS-based service discovery out of the box. No Eureka or Consul is needed: `auth-service.fmp.svc.cluster.local` resolves to the ClusterIP of the auth-service Service object, which kube-proxy load-balances across the pod replicas. The NGINX ingress controller maps the `fmp.local` hostname to the API gateway Service.

3.6 Containerization and Orchestration

Each service has its own multi-stage Dockerfile with a non-root final runtime stage. Docker Compose drives local development; Kubernetes (Minikube during development, ready for any production cluster) drives deployment.

3.7 CI/CD and Automation

Jenkins runs eight pipelines: one root orchestrator and seven per-service jobs (six backend + one frontend). The webhook trigger is `githubPush()`; a `pollSCM('H/5 * * * *')` fallback covers webhook outages. Ansible drives cluster provisioning with four modular roles (`common`, `minikube`, `vault`, `deploy-fmp`).

3.8 Logging and Monitoring

The ELK stack with Filebeat as the DaemonSet log shipper covers observability. Kibana ships with a pre-built dashboard (*FMP-68 — Application Logs*) that is auto-imported on every fresh cluster via a Kubernetes Job.

3.9 Security Tools

HashiCorp Vault (dev mode) provides in-cluster secrets storage. Trivy is architected into the pipeline as the container-image CVE gate. Kubernetes RBAC scopes the Filebeat ServiceAccount to read-only pod/node access. JWT and BCrypt complete the application-layer security posture.

4 Frontend Architecture and Implementation

The frontend is a React 18 single-page application that handles every user-facing concern: sign-in via Google, the dashboard of users and paths, a path-detail view, and the live-tracking map. It is the only component a browser ever talks to directly through the ingress; all backend services are reached transparently through the API gateway.

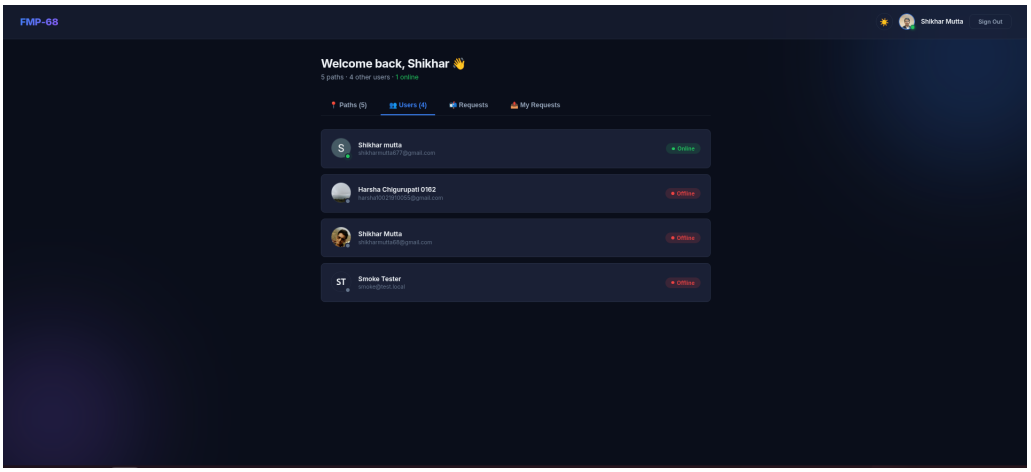


Figure 4: Dashboard — `Users` tab. Each entry shows a presence dot (green = online, red = offline) derived from the users-service `isOnline` flag and pushed live over the WebSocket bus.

4.1 Frontend Architecture Overview

The application is structured around five concepts:

- **Pages.** Top-level route components in `src/pages/`: `LoginPage`, `AuthCallback`, `DashboardPage`, `PathDetailPage`, `LiveTrackingPage`.
- **Contexts.** Cross-cutting state providers in `src/context/`: `AuthContext` (user + token), `ThemeContext` (light/dark), `ToastContext` (notifications).
- **Components.** Reusable UI primitives in `src/components/`: `Navbar`, `PathCard`, `MapView`, `FollowRequestsPanel`, `SentRequestsPanel`, `FollowersList`, `Toast`.
- **Services.** HTTP and WebSocket clients in `src/services/`: `api.js` (Axios wrapper), `socketService.js`, `gpsService.js`, `followRequestService.js`, `followerService.js`.
- **Hooks.** Custom React hooks in `src/hooks/`.

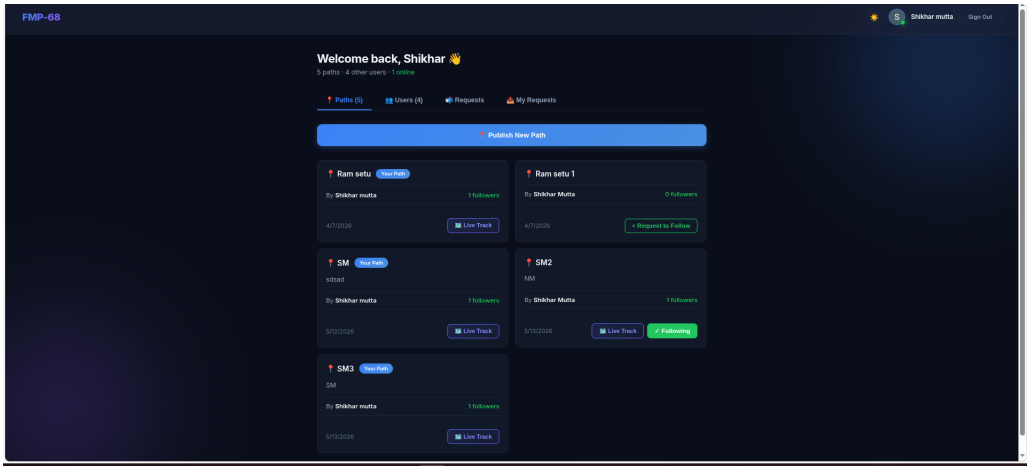


Figure 5: Dashboard — `Paths` tab. Every published path is listed with its owner, follower count, and per-card state: the current user's own paths are tagged *Your Path*, paths the user already follows expose a `Live Track` action with a green *Following* badge, and paths the user does not yet follow show *+ Request to Follow*.

4.2 Communication with Backend

Every API call is an HTTPS request to `REACT_APP_API_URL`, which defaults to `http://localhost:4000` in development and to the cluster ingress `http://fmp.local` in deployment builds. The Axios wrapper attaches the JWT to every outgoing request as `Authorization: Bearer <token>`.

Live tracking uses Socket.io against the same base URL on the `/tracking` namespace. The gateway proxies WebSocket upgrades through to the tracking-service transparently.

4.3 User Interface Flow

Figure 6 traces the canonical user journey from cold sign-in to a live broadcast.

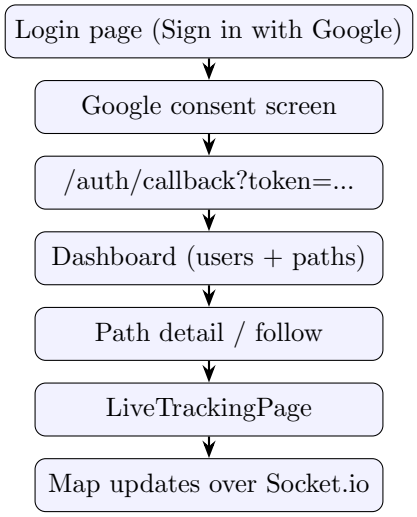


Figure 6: End-to-end UI flow.

4.4 State Management

State is kept local where possible (`useState`, `useReducer`) and lifted into Context only when shared across unrelated subtrees. The three contexts are:

- `AuthContext` — current user, loading flag, `signInWithGoogle()`, `signOut()`, `handleCallback(token)`.
- `ThemeContext` — light/dark theme toggle.
- `ToastContext` — toast queue and dispatcher.

There is no Redux, no Zustand, no React Query. The data model is small enough that this would be overkill.

4.5 Live Tracking Page

`LiveTrackingPage.js` is the most complex screen in the app and handles two distinct flows depending on the role:

Publisher flow. On `Start Tracking`, the page emits `start-tracking` to the WebSocket, opens a `navigator.geolocation.watchPosition()` subscription throttled to one update every three seconds, and emits each new position over `send-location`. `Pause / Resume / End` buttons emit the corresponding control events; tracking state, elapsed time, distance covered, and waypoint count are all derived state.

Follower flow. On `Join`, the page emits `join-tracking` and receives the publisher's existing coordinate trail as a one-shot prefill. Subsequent `location-update` messages append to the publisher's polyline in red. The follower's own GPS trail is broadcast back to the room and drawn in green so the publisher knows who is along. The page computes bearing and distance-to-next-waypoint using the haversine formula so the follower can be guided turn-by-turn.

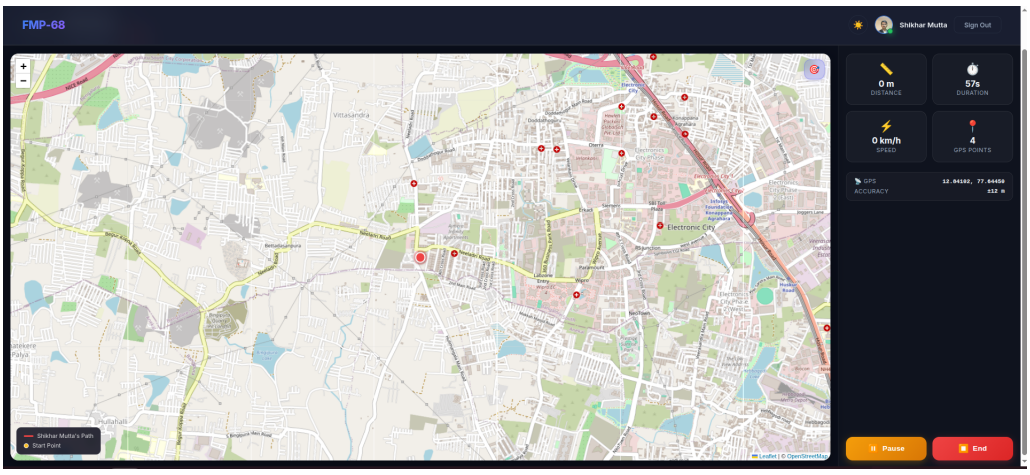


Figure 7: Live Tracking — publisher view during an active recording session. The red polyline is the publisher's GPS trail as emitted over `send-location`; the right-hand telemetry panel shows elapsed time, distance covered, instantaneous speed, and waypoint count, all derived state from the same coordinate stream. The `Pause` and `End` controls emit the matching WebSocket control events.

4.6 Authentication Handling

The JWT is stored as `fmp68_token` in `localStorage`. On application mount, `AuthContext` attempts to validate the token by calling `GET /auth/me`; if the call returns 401, the token is purged and the user is redirected to `/login`. A `PrivateRoute` wrapper guards every non-public route; while the auth check is in flight, a centred spinner is shown rather than a flash of unauthenticated content.

A 30-minute idle timer is wired through a custom hook that listens for `mousemove`, `keydown`, and `touchstart` events; if none fires within 30 minutes, the hook calls `signOut()`.

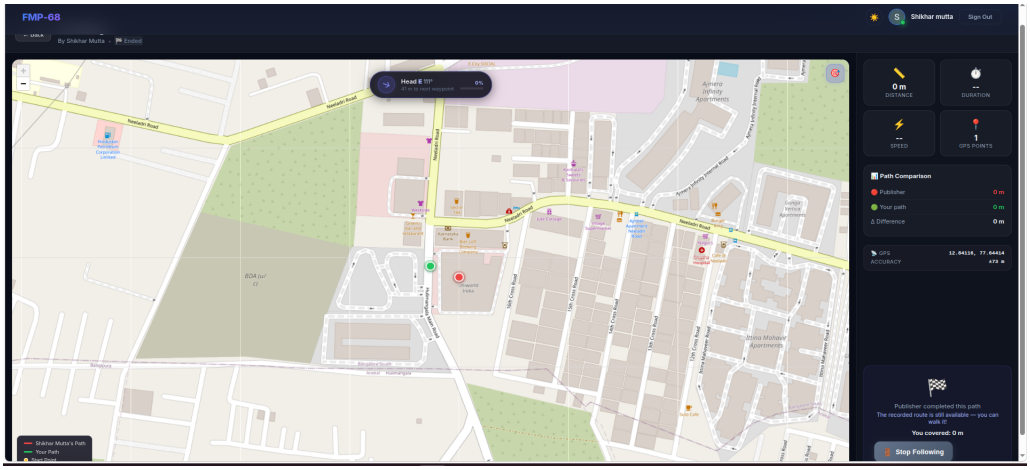


Figure 8: Live Tracking — follower view while navigating a published path. The red polyline is the publisher's recorded trail received on `join-tracking`; the green marker is the follower's own GPS position. The header banner shows the bearing and distance to the next waypoint, computed client-side via the haversine formula so the follower can be guided turn-by-turn.

4.7 Frontend Deployment

The production build is produced by `npm run build` and copied into an Nginx image (Dockerfile multi-stage: builder stage on `node:18-alpine`, runtime stage on `nginx:alpine`). Inside the cluster, the resulting container is exposed through a `frontend` Service and routed by the ingress at the `/` prefix. Rolling updates are zero-downtime because the Nginx pod has a TCP readiness probe on port 80.

5 Backend Architecture and Implementation

The backend is six NestJS 11 services that share a coding style, a language, and an ORM, but have completely independent lifecycles. This section gives the cross-cutting design principles; subsequent sections drill into each service in turn.

5.1 Backend Design Principles

- **One responsibility per service.** Auth-service does OAuth and JWT and nothing else; users-service owns the user document; paths-service owns the path document. No service reaches into another service's database.
- **Stateless containers.** Every replica of a service is interchangeable. Anything that needs to be remembered between requests lives in MongoDB or Redis, never in the process. This is what makes the Kubernetes HPA work.
- **Health-checked everywhere.** Every service exposes `GET /health` returning a 200; Docker Compose healthchecks, Kubernetes readiness probes, and the API gateway's startup wait all key off the same endpoint.
- **Configuration through environment.** Every tunable knob (port, database URL, downstream service URL, secret key) comes from an environment variable. This is what lets the same image run in Compose and in Kubernetes without rebuild.

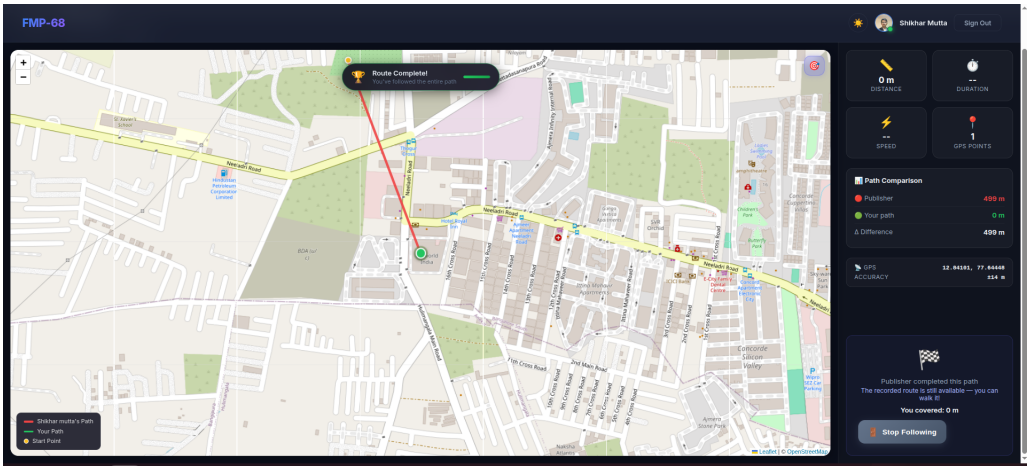


Figure 9: Live Tracking — end-of-session path comparison. When the follower's GPS trail reaches the publisher's last waypoint, a *Route Complete* banner is raised and the *Path Comparison* panel shows publisher distance, follower distance, and the absolute difference, giving an at-a-glance fidelity check on the followed route.

5.2 Backend Service Composition

See Table 1 above for the full inventory. All six services share the following dependencies in their `package.json`:

```
"@nestjs/common": "^11.0.0",
"@nestjs/core":   "^11.0.0",
"@nestjs/config": "^4.0.0",
"@prisma/client": "^5.22.0",
"prisma":         "^5.22.0"
```

5.3 Synchronous Service Communication

Synchronous service-to-service calls happen over plain HTTP. Auth-service, for example, instantiates a small HTTP client (`users-client.service.ts`) that wraps Axios and points at `USERS_SERVICE_URL` (`http://users-service:4002` in Compose, `http://users-service.fmp.svc.cluster.l` in Kubernetes). Identity propagation across the synchronous boundary relies on the gateway: the gateway parses the JWT, verifies the signature, and writes the user id into the `x-user-id` header of the downstream request. Downstream services read that header through a NestJS parameter decorator (`@CurrentUserId()`) rather than re-parsing the JWT themselves.

5.4 Asynchronous Event-Driven Communication

Asynchronous events flow through RabbitMQ. The exchange and queue plumbing is deployed in-cluster; the publisher side is scaffolded in each service's NestJS module but only tracking-service publishes non-trivially (e.g. a `tracking.ended` event when a session finishes). The consumer side is the responsibility of notification-service, which today is a health-only stub.

5.5 Database-per-Service Pattern

Each service has its own Prisma schema file. Even though all four schemas point at the same MongoDB cluster, the language-level boundary is strict: there is no cross-service `import` of a Prisma client. When a service needs data owned by another service, it makes an HTTP call. This is what lets the persistence story evolve toward per-service clusters in the future without rewriting application code.

5.6 Caching Strategy

Redis is used today only by tracking-service, as a short-lived location cache so that a freshly-joined follower can receive the publisher's recent trail without an expensive MongoDB query. Other services do not use Redis; it is available for future use (e.g. JWT revocation lists or rate limiting at the gateway).

5.7 Backend Deployment Model

Each backend service is built into a multi-stage Docker image. The builder stage runs `npm install && npm run build`; the runtime stage copies `dist/` and `node_modules` into a fresh `node:18-alpine` image, sets `USER node`, and exposes the service port. The image is tagged `shikhar68/fmp-<service>:main-<BUILD_NUMBER>` and pushed by Jenkins; Kubernetes pulls it with `imagePullPolicy: Always` when `kubectl set image` updates the Deployment.

6 API Gateway

The API gateway is the single externally reachable HTTP endpoint of the application (modulo the frontend, which is served by its own Nginx container). It is a NestJS service whose only job is to be a smart reverse proxy.

6.1 Role of the API Gateway

- Terminate the public HTTP/WebSocket connection.
- Verify the JWT (when present) and propagate identity into downstream requests as the `x-user-id` header.
- Route requests to the correct backend service based on URL prefix.
- Upgrade WebSocket connections on `/socket.io` and proxy them to tracking-service.
- Apply CORS for the configured `FRONTEND_URL`.

6.2 Routing Mechanism

The gateway uses `http-proxy-middleware` mounted on top of the NestJS Express adapter. Routes are declared imperatively in `main.ts`; the relevant prefixes are listed in Table 2.

Path prefix	Target service
`/auth/*`	auth-service:4001
`/users/*`	users-service:4002
`/paths/*`	paths-service:4003
`/follow-requests/*`	paths-service:4003
`/socket.io/*`	tracking-service:4004 (WS)
`/health`	local — 200 OK

Table 2: API gateway routing table.

6.3 Security Handling

The gateway runs a small middleware that:

1. Reads the `Authorization` header.
2. If a `Bearer` token is present, verifies it against `JWT_SECRET` using `jsonwebtoken.verify`.

3. On success, writes `x-user-id` into the outgoing proxied request.
4. On failure, removes any `x-user-id` header on the outgoing request (defence-in-depth against header injection from upstream).

Note that the gateway does *not* reject unauthenticated requests itself — that is the downstream service's job, via NestJS guards. This keeps the gateway uniform for both public endpoints (login, callback) and private endpoints.

6.4 Service Discovery Integration

The gateway uses Kubernetes' built-in DNS, not a custom service registry. Each downstream is reached at its in-cluster short name (`auth-service:4001` etc.); kube-proxy load-balances across pod replicas, so when the HPA scales the tracking-service from 1 to 3 replicas during a busy live broadcast, the gateway's traffic automatically spreads across the new pods without any rebalancing logic at the gateway layer.

6.5 WebSocket Upgrade

The Socket.io route is the most interesting one. By passing `ws: true` to `createProxyMiddleware` on the `/socket.io` prefix, the gateway transparently upgrades the HTTP CONNECT request into a WebSocket and pipes bytes to and from the tracking-service. Combined with the ingress controller's `proxy-read-timeout: 3600s` annotation, this allows a follower's connection to stay alive for the duration of a multi-hour live broadcast.

6.6 Fault Isolation and Reliability

If a downstream service is unavailable, the gateway returns the upstream's connection failure as a 502 to the client. It does not buffer requests or retry; the client is expected to retry, and the React app does so with exponential backoff on the Axios layer.

6.7 Deployment of API Gateway

The gateway is the only backend service that is reachable from outside the cluster. Its Deployment runs three replicas behind an HPA (`minReplicas=1, maxReplicas=5`, target CPU 70%); its Service is exposed through the ingress at the root path prefix. Its readiness probe is the `/health` endpoint of the gateway itself, not of any downstream service, so a downstream outage does not flap the gateway pods out of the ingress.

7 Service Discovery via Kubernetes DNS

The reference architecture in classic Spring-Cloud microservices uses Eureka or Consul as the service registry. FMP-68 deliberately does not. Kubernetes already provides DNS-based service discovery, and adding a second registry would be redundant infrastructure to operate.

7.1 Why No Eureka?

Eureka's job is to give services a way to find each other's network location without hard-coded IPs. Kubernetes does the same job through three primitives that exist whether you ask for them or not:

- Every `Service` object is given a stable cluster-IP and a DNS name of the form `<svc>.<namespace>.svc.clus`
- The cluster DNS resolver (CoreDNS) resolves that name to the cluster-IP.

- kube-proxy on every node load-balances traffic to the cluster-IP across all Pods backing the Service.

This means a NestJS service that calls `http://users-service:4002/api/v1/users/me` will:

1. Resolve `users-service` via CoreDNS to a stable cluster-IP.
2. Hit kube-proxy, which load-balances to a healthy pod.
3. If users-service scales up or down, or if a pod is replaced, the caller does not notice — the cluster-IP is stable and the backing endpoints are managed by Kubernetes automatically.

7.2 Service Registration

Service registration is implicit: when a Pod's readiness probe passes, the Pod's IP is added to the Service's Endpoints object; when the readiness probe fails or the Pod terminates, the IP is removed. There is no separate registration call from application code.

7.3 Health Monitoring and Fault Handling

Kubernetes performs three kinds of health checks on every Pod:

- **Liveness probe.** If this fails, the Pod is killed and restarted. FMP-68 uses a simple HTTP GET on `/health`.
- **Readiness probe.** If this fails, the Pod is removed from the Service's load-balancing pool. Identical to the liveness probe in this project but conceptually independent.
- **Startup probe (optional).** Used only on notification-service, which has a longer initial RabbitMQ connect time.

7.4 Comparison to Eureka

Kubernetes-native discovery wins on every axis that matters here: zero extra components to run, no separate health-check timer to configure, no client-side library required, and identical semantics across local Compose (DNS based on container name) and Minikube. The only situation in which an external registry would be useful is multi-cluster federation, which is out of scope for this project.

8 Auth Service

The auth-service is the identity provider for the platform. It handles Google OAuth 2.0 sign-in, issues JWTs, and serves the "who am I" and "sign me out" endpoints.

8.1 Core Responsibilities

- Accept Google OAuth 2.0 callback, exchange the authorization code for a Google ID token, extract the user profile.
- Find-or-create the user record by calling users-service.
- Issue a JWT signed with `JWT_SECRET` containing {`sub: userId, email`} and a 7-day expiry.
- Provide `GET /auth/me` for the frontend to validate the token at app boot.
- Provide `POST /auth/signout` that calls users-service to flip `isOnline` to `false`.

8.2 Authentication Workflow

Figure 10 traces the Google OAuth 2.0 sign-in path end-to-end, from the initial browser redirect to the JWT-bearing callback that lands the user back in the SPA.

Figure 10: Google OAuth 2.0 + JWT authentication workflow.

8.3 JWT-Based Security

JWTs are signed using HMAC-SHA256 with `JWT_SECRET`. The payload is intentionally minimal: just `sub` (user id) and `email`. Roles are not encoded in the JWT for this version of the platform — every authenticated user has the same authorisation, and finer-grained ownership checks (e.g. "is this the path's publisher?") happen at the resource level inside paths-service.

The token expires after `JWT_EXPIRES_IN` (default 7 days). The frontend stores the raw token in `localStorage`; the gateway verifies the signature on every request.

8.4 Role-Based Access Control

The platform has a flat permission model in this version: any authenticated user can publish paths, send follow requests, and participate in live tracking. Authorisation that is actually enforced (e.g. "only the publisher of a path may delete it") is *resource-scoped* and lives in paths-service, not in JWT claims.

8.5 Database Design

Auth-service has *no* database of its own. All persistent user state lives in users-service's MongoDB collection. Auth-service is stateless modulo the Google OAuth client credentials, which it reads from environment variables at boot.

8.6 Password Security

There are no passwords in this system. Sign-in is exclusively via Google OAuth; the only secret the service must protect is the JWT signing key, which is supplied through Vault.

8.7 Service Deployment

auth-service runs as a Kubernetes Deployment with one replica by default. It is reachable inside the cluster at `auth-service.fmp.svc.cluster.local:4001` and from outside through

the gateway at `/auth/*`.

8.8 Fault Isolation

If users-service is down, auth-service returns a 502 on `/auth/google/callback` (because find-or-create fails) but `GET /auth/me` continues to succeed as long as the JWT is valid — `/auth/me` does *not* round-trip to users-service for performance reasons. This asymmetry is intentional: it means a brief users-service outage does not log everyone out.

9 Users Service

The users-service owns the canonical `User` document and is the sole writer to the `users` collection.

9.1 Core Responsibilities

- CRUD on user records (create on first Google sign-in, never delete).
- Maintain `isOnline` flag and `lastSeen` timestamp.
- Expose the user directory to the frontend.
- Resolve user IDs in bulk for paths-service's follower lookup.

9.2 Endpoints

- `GET /users` — list of all users with online status, excluding the caller.
- `GET /users/me` — the caller's full profile.
- Internal: `findOrCreate(profile)`, `findManyByIds(ids)`, `setOnlineStatus(userId, isOnline)` — called by other services over HTTP, not exposed at the gateway.

9.3 Database Design

The `User` Prisma model (Listing 1) is the authoritative shape.

Listing 1: Users-service Prisma model.

```
model User {
  id              String   @id @default(auto()) @map("_id") @db.ObjectId
  googleId        String   @unique
  email           String   @unique
  name            String
  picture         String?
  isOnline        Boolean  @default(false)
  lastSeen        DateTime @default(now())
  publishedPaths  Path[]   @relation("publisher")
  followedPathIds String[] @db.ObjectId
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt
}
```

9.4 Online Presence

`isOnline` is flipped to `true` on every successful sign-in (`findOrCreate` sets it) and to `false` on `POST /auth/signout`. A future enhancement is to drive `lastSeen` from WebSocket activity in tracking-service so that the green-dot indicator survives a missed sign-out.

9.5 Service Deployment

users-service runs as a Kubernetes Deployment with one replica. It talks to MongoDB through the `ExternalName` Service `mongo-external`, which resolves to `host.docker.internal:27017` on the developer host.

9.6 Fault Isolation

A users-service outage causes auth-service's `findOrCreate` to fail, so new sign-ins return 502; existing sessions continue to work because the JWT carries the user identity and `/auth/me` does not hit users-service.

10 Paths Service

The paths-service owns the `Path` document and the follow-request workflow.

10.1 Core Responsibilities

- Path CRUD: create, list, view, update, delete.
- Follow / unfollow operations.
- Follower management: list, remove.
- Follow-request workflow: send, approve, reject, cancel.

10.2 Path Endpoints

- `POST /paths` — create a new path (authed).
- `GET /paths` — list all paths (public).
- `GET /paths/published/my-paths` — paths I publish.
- `GET /paths/followed/my-paths` — paths I follow.
- `GET /paths/:id` — fetch one path.
- `POST /paths/:id/follow`, `POST /paths/:id/unfollow`.
- `PUT /paths/:id`, `DELETE /paths/:id` — publisher only.
- `GET /paths/:id/followers` — list followers, enriched with user names and pictures via users-service.
- `DELETE /paths/:id/followers/:followerId` — publisher kicks a follower.

10.3 Follow-Request Endpoints

- `POST /follow-requests` — request to follow a path.
- `GET /follow-requests/pending` — requests on my paths.
- `GET /follow-requests/sent` — requests I sent.
- `POST /follow-requests/approve` — publisher approves.
- `POST /follow-requests/reject` — publisher rejects.
- `DELETE /follow-requests?pathId=...` — cancel my pending request.

10.4 Database Design

Listing 2: Paths-service Prisma model.

```
model Path {
  id            String   @id @default(auto()) @map("_id") @db.ObjectId
  title         String
  description   String?
  publisherId   String   @db.ObjectId
  publisher     User     @relation("publisher", fields: [publisherId], references: [id])
  followerIds   String[] @db.ObjectId
  followRequests String[] @db.ObjectId
  coordinates   Json[]
  status        String   @default("idle")  // idle | recording | paused | ended
  isLive        Boolean  @default(false)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
}
```

10.5 Inter-Service Communication

paths-service calls users-service over HTTP to enrich follower lists and to validate that the requester of a follow exists. It does not write to the `users` collection — that boundary is strictly enforced.

10.6 Reliability and Data Integrity

Follow requests are guarded against double-approval by Prisma's atomic array operations: `push`, `pull`, and `addToSet` run server-side in MongoDB and are race-free. Path deletion is publisher-scoped: the controller compares `x-user-id` (from the gateway) against `path.publisherId` before invoking the repository.

11 Tracking Service

The tracking-service is the realtime spine of the platform. It maintains a Socket.io server on the `/tracking` namespace and broadcasts GPS updates between publisher and followers of a path.

11.1 Core Responsibilities

- Accept WebSocket connections from publishers and followers.
- Manage Socket.io rooms named `path-{pathId}`.
- Persist publisher waypoints to the `Path.coordinates` array and follower waypoints to a `TrackingSession.coordinates` array.
- Maintain the live status of a path (`idle` / `recording` / `paused` / `ended`).
- Broadcast `location-update`, `tracking-started`, `tracking-paused`, `tracking-resumed`, `tracking-ended`, `follower-joined`, `follower-left` events to the room.

11.2 Booking-Style Workflow (Tracking)

The closest analogue in a typical transactional system to a tracking session is the "open transaction" lifecycle. Figure 11 shows the publisher and follower flows side by side.

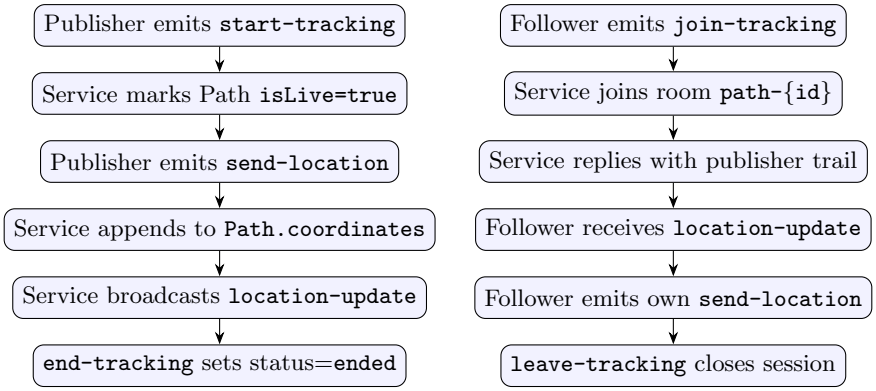


Figure 11: Tracking-session lifecycle, publisher (left) and follower (right).

11.3 Database Design

Listing 3: Tracking-service Prisma model.

```
model TrackingSession {
  id          String   @id @default(auto()) @map("_id") @db.ObjectId
  pathId      String   @db.ObjectId
  userId      String   @db.ObjectId
  role        String   // "publisher" | "follower"
  coordinates Json[]   // [{lat, lng, timestamp}, ...]
  startedAt   DateTime @default(now())
  endedAt     DateTime?
  isActive    Boolean  @default(true)
}
```

11.4 Caching and Seat-Locking Analogue

A booking-system uses Redis to lock a seat against double-booking; the tracking analogue is to cache the publisher's recent trail in Redis so a newly-joined follower can be served the last N coordinates without hitting MongoDB. The Redis key shape is:

```
trail:{pathId} -> JSON array of last 200 coordinates (TTL 1 hour)
```

11.5 Asynchronous Messaging

On `end-tracking`, tracking-service publishes a `tracking.ended` event to the `fmp.events` RabbitMQ exchange. Notification-service is the intended consumer for this event (to send a "your tracking session is saved" email).

11.6 Service Integration

tracking-service reads from paths-service over HTTP only at the moment a path's metadata is needed (e.g. `get-tracking-data` to return title + publisher for the map header). All hot-path operations stay inside tracking-service.

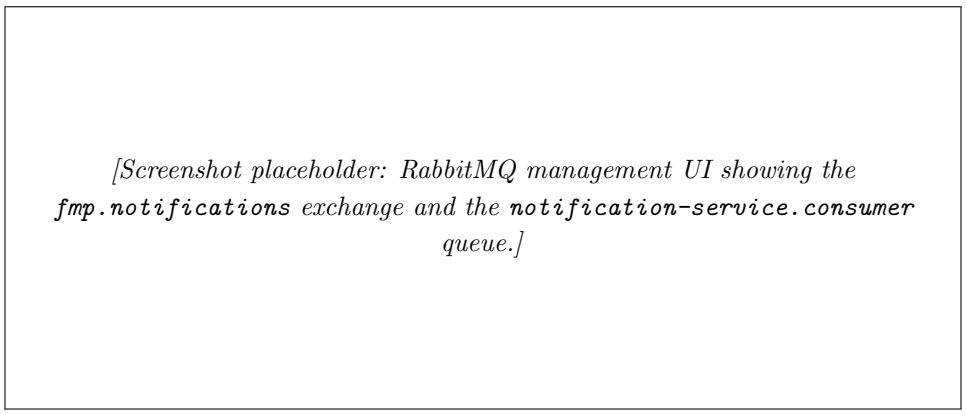
11.7 Deployment Architecture

tracking-service runs as a Kubernetes Deployment with an HPA targeting 70% CPU and a range of one to four replicas. Because Socket.io sessions are sticky, the ingress is configured with `nginx.ingress.kubernetes.io/affinity: cookie` so a given follower remains pinned to one tracking-service pod for the duration of their broadcast.

11.8 Reliability and Consistency

Coordinate writes are append-only and idempotent at the application level (clients include a timestamp that the server uses to deduplicate). A pod restart during a live session causes the followers' Socket.io clients to reconnect automatically; the publisher's next `send-location` re-broadcasts the current position, so the gap is invisible to the user.

12 Notification Service



12.1 Responsibilities

notification-service is the asynchronous fan-out point for any user-facing event that should generate an email or a push notification. In its target state it consumes from RabbitMQ and forwards to an SMTP transport.

12.2 Current Implementation Status

notification-service is *scaffolded but not wired*. It boots, exposes `GET /health`, and connects to RabbitMQ on startup so the broker considers it a registered consumer; however, the message handler that would parse a `tracking.ended` payload and generate an email is not yet implemented. This is the single unfinished piece of application logic in the project, and is documented honestly here rather than glossed over.

12.3 Event-Driven Workflow (Target State)

1. Publisher service emits an event (e.g. `tracking.ended`) to the `fmp.events` exchange.
2. RabbitMQ routes the event to the `notification-service.queue` bound to the exchange.
3. notification-service consumes the event.
4. Service enriches the event by calling paths-service and users-service over HTTP (to fetch path title and follower emails).
5. Service sends one email per recipient via the configured SMTP transport.

12.4 Email Delivery

Email transport is intended to be plain SMTP (e.g. Gmail with an application password) supplied via Vault under `secret/fmp/smtp`. The pluggability is by design: swapping SMTP for SendGrid is a one-file change.

12.5 Asynchronous and Reliable Processing

RabbitMQ provides at-least-once delivery semantics; the consumer is expected to be idempotent. Acknowledgements are manual, so a crashed consumer redelivers the message to a replacement pod.

12.6 Deployment Architecture

notification-service runs as a Kubernetes Deployment with one replica. It is not reachable from outside the cluster: there is no ingress route mapped to it, and no peer service calls it over HTTP.

12.7 Security and Isolation

SMTP credentials would be sourced from Vault, never from a Kubernetes Secret directly. The pod runs as a non-root user and has no privileged capabilities.

13 Redis

[Screenshot placeholder: Redis CLI output showing the `trail:*` keys with TTLs, demonstrating the live-broadcast cache layer.]

13.1 Role of Redis in the Tracking Workflow

Redis is the short-lived cache used by tracking-service to give a freshly joining follower the last few hundred coordinates of a live broadcast without a MongoDB round-trip. The dataset is intentionally small, write-heavy, and ephemeral — exactly Redis's strengths.

13.2 Why Redis Is Suitable

- Microsecond reads and writes for in-memory data.
- Built-in TTL means stale tracks expire without a sweeper job.
- Single-threaded execution model gives atomic `LPUSH` / `LTRIM` for trail trimming.
- Zero configuration in Kubernetes.

13.3 Key Schema

```
trail:{pathId}   -> LIST of recent JSON coordinates, capped at 200
                    LTRIM trail:{pathId} 0 199 after each LPUSH
                    EXPIRE trail:{pathId} 3600  on every write
```

13.4 Data Volatility and Safety

Redis is configured *without* persistence (no AOF, no RDB snapshots). This is a deliberate choice: the authoritative coordinate log is in MongoDB; Redis is only a read-side accelerator. A Redis restart causes a few seconds of slightly slower follower-join responses while the cache repopulates, and nothing more.

13.5 Deployment in Kubernetes

Redis runs as a single-replica Deployment in the `fmp` namespace, exposed through a ClusterIP Service on port 6379. No PersistentVolume is mounted; the pod restarts with empty memory.

13.6 System Reliability Contribution

The cache is non-essential: tracking-service will degrade gracefully to direct MongoDB queries if Redis is unreachable. A circuit-breaker in the cache wrapper trips after three consecutive errors and stays open for thirty seconds.

14 RabbitMQ Messaging Backbone

RabbitMQ is the asynchronous backbone of the platform. It is deployed in-cluster and used for event-driven fan-out between services that should not block each other.

14.1 Why a Broker?

A synchronous HTTP call from tracking-service to notification-service would couple their availabilities: an email outage would slow down the tracking session. Pushing the event into a queue lets tracking-service finish its work in microseconds and lets notification-service drain the queue at its own pace.

14.2 Topology

- One *topic* exchange: `fmp.events`.
- One queue per consumer service: `notification-service.queue` bound with routing key `tracking.*`.
- Management UI exposed at port 15672 (guest/guest in dev).

14.3 Deployment

RabbitMQ runs as a single-replica Deployment in the `fmp` namespace with a TCP readiness probe. No clustering or persistent volume is configured for the dev environment, which is sufficient because no event is critical enough to require durability beyond a pod restart in the current functional scope.

14.4 Operational Visibility

The management UI is invaluable during demos: queue depth, consumer count, publish rate, and message inspection all live there. Logs from the broker are picked up by Filebeat with the rest of the cluster and are queryable in Kibana under `service: rabbitmq`.

15 Database Layer

15.1 Service-Specific Schemas

Each NestJS service has its own Prisma schema file, even though all schemas point to the same MongoDB cluster. The collections owned per service are:

- users-service → `users`
- paths-service → `paths`
- tracking-service → `tracking_sessions`

auth-service, api-gateway, and notification-service have no schema.

15.2 MongoDB Deployment Model

MongoDB runs *outside* the Kubernetes cluster, on the developer host. This is a deliberate simplification for the academic context: running a replicated stateful database inside Minikube would be disproportionate to the project's scope. Inside the cluster, MongoDB is reached through a Kubernetes `Service` of type `ExternalName` that resolves `mongo-external` to `host.docker.internal`.

15.3 Replica Set Requirement

Prisma's MongoDB connector requires a replica set, not a standalone node. The project documents the `rs.initiate()` setup in the README. In a production deployment, this would be replaced by a managed MongoDB Atlas cluster or a self-hosted three-node replica set inside the cluster.

15.4 Secure Credential Management

Database credentials are sourced from Vault at boot via the `secret/fmp/db` path. In the current dev mode, the credentials are empty (the host MongoDB does not require authentication); in production the same code path picks up real credentials from Vault without any application change.

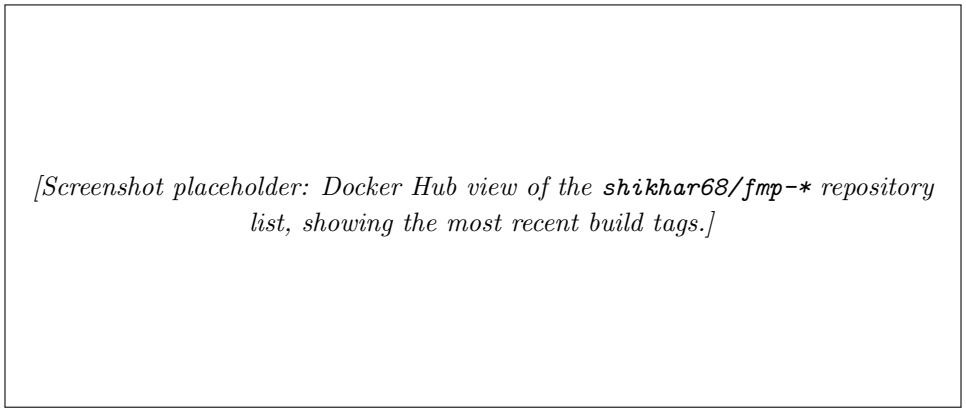
15.5 Testing with In-Memory Alternatives

The Jenkins `Install & Test` stage runs `npm test --passWithNoTests`. Where unit tests do exist, they use Prisma's testing mode (an in-memory PostgreSQL or mocked client) so they do not need a live MongoDB. Adding more tests is the largest single area of technical debt and is called out in the Testing section.

15.6 Consistency and Reliability

Application-level consistency relies on MongoDB's atomic operators (`$push`, `$pull`, `$addToSet`) for array fields and on per-document atomicity for everything else. The project does not use multi-document transactions because the data model is intentionally designed so that no business invariant requires them.

16 Docker Containerization



16.1 Per-Service Dockerfiles

Every backend service has a multi-stage Dockerfile. The two-stage pattern is:

Listing 4: Two-stage Dockerfile pattern (auth-service).

```
# ---- builder ----
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# ---- runtime ----
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist          ./dist
COPY --from=builder /app/package.json  ./
USER node
EXPOSE 4001
CMD ["node", "dist/main"]
```

The runtime image is small (Alpine base, no build toolchain) and runs as the non-root `node` user, which trims attack surface in line with the DevSecOps brief.

16.2 Frontend Dockerfile

The frontend uses the same two-stage pattern but with an `nginx:alpine` runtime stage that serves the React build artefacts. A custom `nginx.conf` enables SPA fallback (every unknown path returns `index.html` so React Router can take over) and compression.

16.3 Docker Compose Setup

Local development uses two Docker Compose files joined by an external network `fmp-net`:

- `backend/docker-compose.yml` — all six microservices, RabbitMQ, Redis. MongoDB is reached via `host.docker.internal`.
- `frontend/docker-compose.yml` — the Nginx container, attached to `fmp-net` as an external network.

16.4 Networking and Service Access

All compose services join `fmp-net`, so services find each other by their Compose service name (`auth-service`, `users-service`, ...). The same names are reused as Kubernetes Service names, which means the same image runs in both environments without configuration changes.

16.5 Environment Profiles

Each service reads its configuration from `.env` files. The template `.env.example` files are checked into the repo with placeholder values; real values are supplied at runtime through either Compose's `env_file` or Kubernetes Secrets / Vault.

16.6 Resource Control

The backend Compose file sets per-service CPU and memory limits (`deploy.resources.limits.cpus: '0.5'`, `deploy.resources.limits.memory: 512M`). This is mirrored in the Kubernetes manifests.

16.7 Clean-Rebuild Workflow

The root `start.sh` script always performs a clean rebuild (`docker compose build --no-cache`). This costs a few minutes on a cold machine but guarantees that the running containers were produced from the current working-tree source — exactly the property a DevOps platform must give you.

17 Jenkins and CI/CD Automation

[Screenshot placeholder: Jenkins dashboard showing the orchestrator and seven per-service pipelines, with their last build status.]

17.1 Overall CI/CD Architecture

The CI/CD system is two-tiered. The root orchestrator pipeline (`backend/Jenkinsfile`) is triggered on every push. It diffs the latest commit against its parent, maps the changed file paths to per-service Jenkins job names, and fans out to those jobs in parallel. A separate "infra changed" branch covers Kubernetes manifest changes by running `kubectl apply` directly.

Figure 12 captures the full data flow.

17.2 Trigger Configuration

Every pipeline declares the same trigger block:

Listing 5: Trigger configuration used by all pipelines.

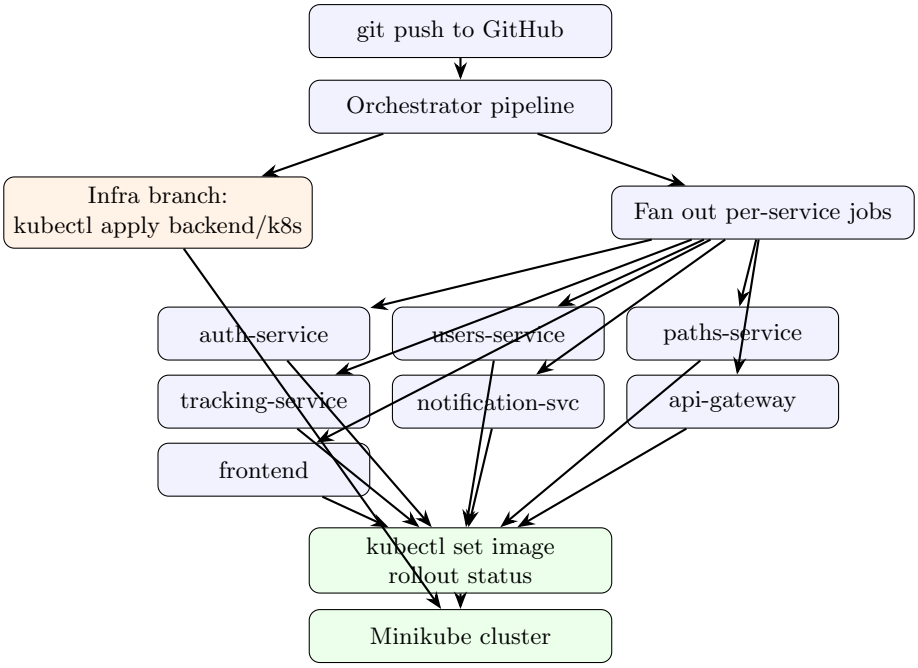


Figure 12: Jenkins CI/CD topology.

```
triggers {
    githubPush()
    pollSCM('H/5 * * * *')   // 5-minute fallback for webhook outages
}
options {
    disableConcurrentBuilds(abortPrevious: true)
    timeout(time: 25, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
}
```

`abortPrevious: true` means that if two pushes arrive in quick succession, the older build is killed and the newer one starts — which is what you want for a deployment pipeline.

17.3 Selective Build Logic

The orchestrator's `Detect Changes` stage runs:

```
git diff --name-only HEAD~1 HEAD
```

then matches each line against a regex per service:

```
backend/api-gateway/          -> job: fmp-api-gateway
backend/auth-service/         -> job: fmp-auth-service
backend/users-service/        -> job: fmp-users-service
backend/paths-service/        -> job: fmp-paths-service
backend/tracking-service/     -> job: fmp-tracking-service
backend/notification-service/-> job: fmp-notification-service
backend/k8s/                  -> infra branch
frontend/                     -> job: fmp-frontend
```

Jobs whose folder did not change are skipped; jobs that did fire in parallel. A pure docs commit hits the *Nothing Changed* branch and exits in seconds — this is one of the project's explicit innovations.

17.4 Per-Service Pipeline Stages

Every per-service pipeline runs the same stage sequence (see Figure 13):

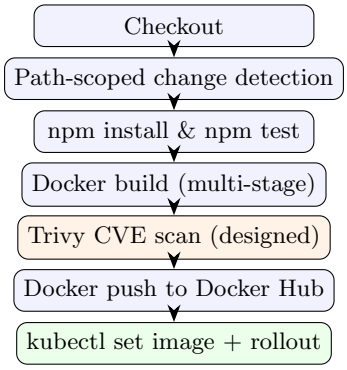


Figure 13: Per-service Jenkins pipeline stages. Orange stage is the DevSecOps gate.

The `Path-scoped change detection` stage is what makes the orchestrator's selective rebuild robust: even if Jenkins accidentally triggers a service whose folder didn't change (e.g. via the `pollSCM` fallback), the per-service pipeline itself will detect that and skip.

17.5 Selective Build and Parallel Deployment

Selective builds save build time and Docker Hub bandwidth. A single-service edit fans out to exactly one pipeline; a six-service refactor fans out to six pipelines that all run in parallel.

17.6 Automated Kubernetes Deployment

The deployment step is two commands:

```
kubectl set image deployment/${SERVICE_NAME} \
        ${SERVICE_NAME}=${IMAGE_REMOTE} -n ${K8S_NAMESPACE}
kubectl rollout status deployment/${SERVICE_NAME} \
        -n ${K8S_NAMESPACE} --timeout=180s
```

`rollout status` blocks until the new pods are ready, so a failure here fails the Jenkins build and surfaces the regression loudly.

17.7 Testing Integration

The `Install & Test` stage runs `npm test -- --passWithNoTests --coverage`. Test suites that exist are picked up by Jest and reported; services with no tests yet do not block the build, but their absence is visible in the Jenkins console output.

17.8 Benefits of the CI/CD Pipeline

- Single push produces a deployed cluster update in roughly 2–3 minutes (single-service change).
- Parallel fan-out means a six-service change still finishes in 4–5 minutes.
- Selective rebuild keeps unrelated services on their existing production images, reducing churn.
- Every deployed image is traceable to a Git SHA via the build number tag.

18 Kubernetes Orchestration

[Screenshot placeholder: Output of `kubectl get pods -A` after a clean `ansible-playbook` run, showing all pods in Running / Ready state across the fmp, elk, and vault namespaces.]

18.1 Role of Kubernetes in the System

Kubernetes orchestrates every workload: it schedules pods, restarts unhealthy containers, balances traffic across replicas, manages storage volumes, and applies rolling updates with zero downtime. In this project, the cluster is Minikube running on the developer host; the same manifests apply unchanged to any production-grade cluster (EKS, GKE, AKS).

18.2 Namespaces and Logical Isolation

The cluster is partitioned into three namespaces:

- `fmp` — application + ingress + Redis + RabbitMQ.
- `elk` — Elasticsearch, Logstash, Kibana, Filebeat.
- `vault` — HashiCorp Vault.

Namespaces are the unit of RBAC scoping and resource-quota application in Kubernetes. Separating concerns this way means the observability stack can be torn down and re-applied without touching the application pods, and vice versa.

18.3 Workload Types Used

- `Deployment` — every stateless service (all six backend services, the frontend Nginx, Kibana, Logstash, Vault).
- `StatefulSet` — Elasticsearch (because indices are tied to a stable pod identity).
- `DaemonSet` — Filebeat (one shipper per node).
- `Job` — Kibana dashboard import (one-shot post-deploy task).
- `Service` (ClusterIP) — inter-service connectivity.
- `Service` (ExternalName) — MongoDB.
- `Ingress` — `fmp.local` entry point.
- `ConfigMap`, `Secret` — configuration plumbing.
- `HorizontalPodAutoscaler` — api-gateway, tracking-service.
- `ServiceAccount`, `ClusterRole`, `ClusterRoleBinding` — Filebeat RBAC.

18.4 Service Discovery and Networking

Discussed in detail in Section 7. Every service has a stable DNS name and is load-balanced by kube-proxy across its pod replicas.

18.5 Configuration and Secrets Management

Non-secret configuration lives in ConfigMaps; secret configuration lives in Vault. Kubernetes Secrets are used only as a thin wrapper where direct Vault integration is too heavy (e.g. ingress TLS), and in those cases the Secret is created *by* the Vault postStart hook rather than checked into Git.

18.6 Resource Management and Stability

Every Deployment declares CPU and memory `requests` and `limits`. Typical values:

```
resources:
  requests: { cpu: 100m,  memory: 128Mi }
  limits:   { cpu: 500m,  memory: 512Mi }
```

This lets the kube-scheduler bin-pack pods rationally and prevents a runaway container from starving its neighbours.

18.7 Self-Healing and Reliability

Kubernetes will:

- Restart a container that exits non-zero (CrashLoopBackOff).
- Mark a pod unready if its readiness probe fails, removing it from the Service's load-balancing pool.
- Kill and replace a pod whose liveness probe fails.
- Reschedule a pod whose node becomes unreachable.

No application code is required for any of these behaviours; they are emergent properties of correctly-declared probes and replica counts.

18.8 Scalability

Section 19 (Ingress and External Access) and Section 20 (HPA) go deeper. In short, every stateless service can be scaled by increasing its replica count or by raising its HPA ceiling.

18.9 Production Readiness

The same manifests deploy unchanged to any Kubernetes distribution. The Minikube-specific assumptions are isolated to:

- `mongo-external` (would become a real Service in production).
- `NodePort` services (would become `LoadBalancer` or stay behind the ingress).

19 Ingress and External Access

The cluster has exactly one externally reachable endpoint: the NGINX ingress controller, listening on `fmp.local`.

19.1 Ingress Controller

The NGINX ingress controller is enabled in Minikube with `minikube addons enable ingress`. The Ansible `minikube` role automates this. Once enabled, the controller watches every `Ingress` object in the cluster and rewrites its own NGINX configuration on the fly to match.

19.2 Hostname-Based Routing

`fmp.local` is the externally visible hostname. The developer maps it to the Minikube IP via `/etc/hosts`:

```
$(minikube ip) fmp.local
```

This name is what the React production build embeds as `REACT_APP_API_URL=http://fmp.local`.

19.3 Path-Based Routing

Path prefix	Backend
`/`	frontend Service (Nginx)
`/auth/*`	api-gateway:4000
`/users/*`	api-gateway:4000
`/paths/*`	api-gateway:4000
`/follow-requests/*`	api-gateway:4000
`/socket.io/*`	api-gateway:4000 (WebSocket)

Table 3: Ingress path-based routing.

The frontend gets `/`; everything else flows through the API gateway, which then dispatches internally.

19.4 WebSocket Annotations

For the Socket.io route to survive long-lived broadcasts, the ingress carries the annotations:

```
nginx.ingress.kubernetes.io/proxy-read-timeout:    "3600"
nginx.ingress.kubernetes.io/proxy-send-timeout:    "3600"
nginx.ingress.kubernetes.io/affinity:              "cookie"
nginx.ingress.kubernetes.io/session-cookie-name:   "fmp-track"
```

Affinity is important because Socket.io expects the same backend pod to handle every frame of a session.

19.5 Security and Isolation

Only the ingress controller is reachable from outside the cluster; every other service has a ClusterIP and is invisible to the host. The ingress is HTTP-only in dev; a TLS termination story is left as future work and is one of the documented gaps.

19.6 Scalability and Load Balancing

The ingress controller itself runs as a Deployment with multiple replicas and is fronted by Minikube's tunnel. Behind it, NGINX load-balances to each Service's ClusterIP, and kube-proxy load-balances to the backing pods. This gives two layers of L4/L7 load balancing essentially for free.

19.7 Operational Benefits

- One host, one TLS certificate (when we add TLS).
- Internal services have no public surface — defence in depth.
- Routing rules live in code (`ingress.yaml`) and are applied by `kubectl apply` like everything else.

20 Horizontal Pod Autoscaling

Two services in this project carry an HPA: the api-gateway (which sees every external request) and the tracking-service (whose CPU demand spikes during a live broadcast with many followers).

20.1 HPA Configuration

Listing 6: api-gateway HPA.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-gateway
  namespace: fmp
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-gateway
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

20.2 How HPA Works

The metrics-server samples CPU usage every 15 seconds. If the average CPU across all pods in the Deployment crosses 70%, the HPA controller increases the replica count; if it sits below the threshold for the configured stabilisation window, it scales back down.

20.3 Why These Two Services?

- **api-gateway** — every request goes through it. CPU scales with request rate, not with payload size, because JWT verification is the dominant cost.
- **tracking-service** — Socket.io fan-out is CPU-bound. A path with fifty followers means the publisher's single send produces fifty broadcast emits, all on one pod unless we scale out.

20.4 Why Not the Other Services?

auth-service, users-service, paths-service, and notification-service all have request profiles that are flat and small. Auth handling is dominated by the OAuth round-trip with Google; users and paths are straightforward CRUD. Adding HPA to them would be premature optimisation.

21 Ansible Configuration Management

This project takes the *opposite* position to the reference report's "Why Ansible Is Not Required" section. Ansible is *exactly* what is required, and the project carries a four-role playbook to prove it.

21.1 Why Ansible Here

Kubernetes is good at running pods, but it does not provision the cluster itself, install `kubectl`, start Minikube, or seed Vault from environment variables. Those are exactly the jobs Ansible exists to do. Saying "Kubernetes is enough" is true only if you assume the cluster already exists and the tooling on the host is already configured — which is precisely what a fresh-machine demo cannot assume.

21.2 Playbook Structure

`backend/ansible/site.yml` is a single playbook that runs four roles in order:

```
- hosts: localhost
  connection: local
  become: false
  roles:
    - common      # docker, kubectl, helm
    - minikube    # start cluster, enable ingress + metrics-server
    - vault       # deploy Vault, seed secrets
    - deploy-fmp  # apply backend/k8s manifests, wait for Ready
```

21.3 Role: common

Installs the host-side toolchain idempotently: Docker, `kubectl`, `helm`. Skips installation if the binary is already present at the expected version, which keeps re-runs fast.

21.4 Role: minikube

Starts the Minikube cluster with a sane resource profile (4 CPU, 8 GiB), enables the `ingress` and `metrics-server` addons, and waits for the control plane to become responsive.

21.5 Role: vault

Applies `backend/k8s/vault/` manifests and waits for the Vault pod to pass its readiness probe. Vault's own seed script runs via the `postStart` hook and populates `secret/fmp/{auth,db,rabbitmq}` from the environment variables passed to the Vault Deployment.

21.6 Role: deploy-fmp

Applies the application manifests in dependency order:

1. Namespace, ConfigMap, Secret, MongoDB ExternalName.
2. RabbitMQ, Redis (infrastructure).
3. Per-service Deployments + Services + HPAs.
4. Ingress.
5. ELK stack and Kibana import Job.

Each step waits for the previous one to be `Available` via `kubectl rollout status` so the playbook does not race ahead of pod readiness.

21.7 Idempotency

Ansible's value is that running the playbook twice produces the same state. Every task in the playbook uses Ansible's `kubernetes.core.k8s` module (which is itself idempotent via `kubectl apply`'s server-side merge) or a `when:` guard that skips already-done work. A re-run after a working install is a no-op.

21.8 Inventory

The inventory is trivial: a single local host. This is a single-developer-machine playbook, not a fleet-management playbook. Scaling to a fleet would require adding remote hosts and SSH credentials but no structural change.

22 HashiCorp Vault — Secrets Management

[Screenshot placeholder: Vault UI showing the kv-v2 secrets engine mounted at `secret/` with the `fmp/auth`, `fmp/db`, and `fmp/rabbitmq` paths.]

22.1 Why Vault?

Kubernetes Secrets are base64-encoded, not encrypted at rest by default. Vault gives:

- Encryption at rest with key rotation.
- Versioned secrets (kv-v2).
- Fine-grained access policies.
- Audit logs of every read.

For a DevSecOps project, all four matter.

22.2 Deployment Mode

Vault runs in *dev mode* for this project: `vault server -dev -dev-root-token-id=fmp-dev-root`. Dev mode auto-unseals on every boot and stores data in memory only. This is explicitly not production-grade, but it gives a working secrets workflow without operating a separate database for Vault's storage backend.

22.3 Auto-Seeding via postStart

Dev-mode Vault starts empty. To make secrets available at pod startup, the Vault Deployment carries a Kubernetes `lifecycle.postStart` hook that runs an in-pod shell script the moment

the container is up:

Listing 7: Vault auto-seed (paraphrased).

```
lifecycle:
  postStart:
    exec:
      command:
        - sh
        - /vault/seed/seed.sh

# seed.sh
vault kv put secret/fmp/auth \
   jwt_secret="${FMP_JWT_SECRET}" \
   google_client_id="${FMP_GOOGLE_CLIENT_ID}" \
   google_client_secret="${FMP_GOOGLE_CLIENT_SECRET}"

vault kv put secret/fmp/db \
   mongodb_uri="mongodb://mongo-external:27017/fmp68?replicaSet=rs0&directConnection=true"

vault kv put secret/fmp/rabbitmq \
   url="amqp://guest:guest@rabbitmq.fmp.svc.cluster.local:5672"
```

This is the project's signature innovation in the secrets layer: no manual `vault kv put` after a cluster rebuild, no out-of-band state file. Re-deploying the cluster re-seeds Vault automatically.

22.4 Application Integration

Services fetch their secrets at boot via the Vault HTTP API:

```
GET http://vault.vault.svc.cluster.local:8200/v1/secret/data/fmp/auth
Headers: X-Vault-Token: ${VAULT_TOKEN}
```

The token is supplied through a Kubernetes Secret that lives only on the pod's filesystem; the real secret payloads (`JWT_SECRET`, `GOOGLE_CLIENT_SECRET`, ...) never appear in any manifest.

22.5 Production Path

Moving from dev to production requires:

1. Replacing `-dev` with a real storage backend (Raft or Consul).
2. Configuring auto-unseal (KMS, Transit, etc.).
3. Replacing the postStart seed with a Vault Agent Injector sidecar pattern.
4. Defining real `Policies` per ServiceAccount.

None of the application code changes; the integration points are already correct.

23 Trivy — DevSecOps CVE Gate

23.1 Role of Trivy in the Pipeline

Trivy is architected into every per-service Jenkins pipeline as the container-image vulnerability scanner. It runs *after* the local Docker build and *before* the Docker Hub push, so an image with a HIGH or CRITICAL CVE is identified before it leaves the build host.

23.2 Pipeline Position

The Trivy stage sits between `Docker Build` and `Docker Push` (refer to Figure 13). This position is what makes it a real gate: a failed scan can block the push.

23.3 Configuration (Designed)

The intended stage definition is:

```
stage('Security Scan (Trivy)') {
  steps {
    sh '''
      trivy image \
        --severity HIGH,CRITICAL \
        --ignore-unfixed \
        --exit-code 0 \
        --format table \
        ${IMAGE_LOCAL}
    '''
  }
}
```

23.4 Educational vs Enforcing Mode

The `--exit-code 0` setting prints findings without failing the build. This is intentional for the educational phase of the project: a fresh `node:18-alpine` base image typically carries a handful of HIGH CVEs that would block every build.

The *flip to enforcing mode* is a single character change: `--exit-code 1`. Once the base image is hardened and the project is willing to fail builds on CVE findings, this is the only edit needed.

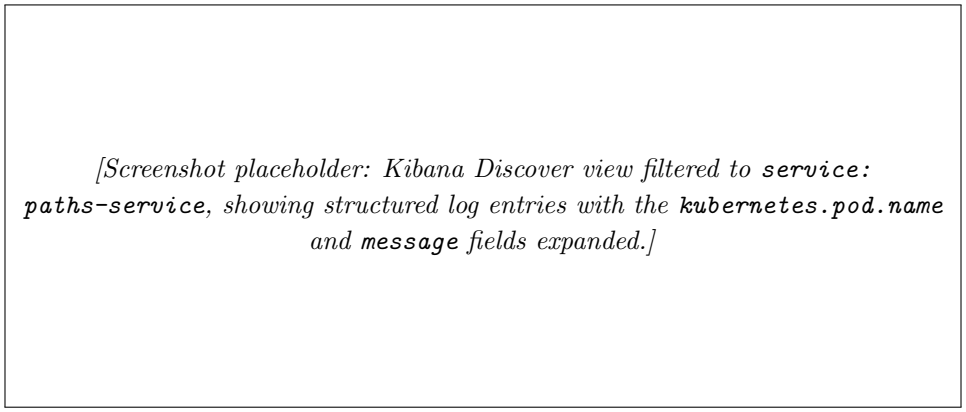
23.5 Honest Status

At the time of writing, the Trivy stage is part of the platform design narrative but has not yet been merged into the per-service Jenkinsfiles. The README presents it as the project's central DevSecOps innovation, so adding it concretely is the first follow-up work item. This honesty about implementation status is itself part of the DevSecOps discipline — the gap is in the issue tracker, not hidden in a slide.

23.6 Why HIGH and CRITICAL Only

Trivy categorises CVEs into UNKNOWN / LOW / MEDIUM / HIGH / CRITICAL. HIGH and CRITICAL cover essentially all CVEs with a working public exploit; MEDIUM and below are typically operational hardening recommendations. Scoping the gate to HIGH and CRITICAL gives a tight signal-to-noise ratio for a developer-facing pipeline.

24 ELK Stack for Centralized Logging



24.1 Purpose of Centralized Logging

Across the `fmp` namespace there are seven or more pods at any time. Manually `kubectl logs`-ing across them is unworkable. The ELK stack aggregates every container's stdout/stderr into one searchable, time-indexed corpus.

24.2 Components

- **Elasticsearch** — single-node StatefulSet, port 9200.
- **Logstash** — Deployment with a Beats input on port 5044 and an Elasticsearch output.
- **Filebeat** — DaemonSet, one pod per node.
- **Kibana** — Deployment, NodePort exposed on 5601.

All four live in the `elk` namespace.

24.3 Filebeat

Filebeat runs as a DaemonSet and mounts the host paths where the Kubernetes runtime writes container logs:

```
volumeMounts:
  - mountPath: /var/log/pods
    name: varlogpods
    readOnly: true
  - mountPath: /var/lib/docker/containers
    name: varlibdockercontainers
    readOnly: true
```

It enriches each log line with Kubernetes metadata (`pod.name`, `namespace`, `container.name`, `labels`) and forwards to Logstash on port 5044.

24.4 Filebeat RBAC

The DaemonSet runs as a dedicated ServiceAccount (`filebeat-sa`) bound to a tightly scoped ClusterRole that grants *only* the verbs Filebeat needs:

- `get, list, watch` on `pods`, `namespaces`, `nodes`.
- Nothing else — no write, no delete, no other resources.

This least-privilege RBAC posture is the second DevSecOps highlight on the observability side.

24.5 Logstash

Logstash's pipeline does two important things:

1. Parse JSON-formatted log messages (NestJS Pino logger emits JSON when configured) and promote the embedded fields to top-level Elasticsearch fields.
2. Promote `kubernetes.container.name` into a top-level `service` field, so Kibana dashboards can pivot by service name without per-pod awareness.

24.6 Elasticsearch Index Pattern

Logs are indexed under the daily pattern `fmp-logs-YYYY.MM.dd`, which makes index lifecycle management trivial: drop old indices to reclaim disk.

24.7 Kibana Dashboards as Code

The headline observability innovation is that the Kibana dashboard is checked into Git and re-applied on every fresh cluster. The mechanism:

1. The saved objects (index pattern + four visualisations + one dashboard) are exported as NDJSON to `backend/k8s/elk/kibana-objects.ndjson`.
2. A Kubernetes `Job` (`kibana-import-job.yaml`) waits for Kibana's `/api/status` to report `green`, then POSTs the NDJSON to `/api/saved_objects/_import?overwrite=true`.
3. On any fresh cluster, the dashboard "FMP-68 — Application Logs" appears automatically.

24.8 Dashboard Contents

Panel	Type	Source
Total Log Events	Metric	`count(*)` over `fmp-logs-*`
Log Volume Over Time	Stacked histogram	`@timestamp` × `service.keyword`
Logs by Service	Donut chart	`service.keyword`
Top Services Table	Sorted table	`service.keyword` by count

Table 4: Panels in the auto-imported Kibana dashboard.

24.9 Operational Benefits

- One Kibana URL, every service's logs.
- Free-text search across the entire fleet.
- Time-correlated incident debugging.
- Per-service log volume monitoring catches a chatty service before it floods disk.

24.10 Production Readiness Gaps

The dev ELK stack is single-node and unauthenticated. Production needs:

- Multi-node Elasticsearch cluster (3 master, 3 data minimum).
- Kibana behind authentication (e.g. OIDC).
- Index Lifecycle Management policies for retention.
- Persistent volumes for index storage.

25 Testing Strategy

25.1 Honest Status

Tests are the largest area of technical debt in this project. The CI pipeline runs `npm test --passWithNoTests` so missing tests do not break the build, but the testing pyramid is sparse and documented here *as a plan* rather than as completed work.

25.2 Unit Testing

Each NestJS service is set up with Jest (NestJS's default) and `@nestjs/testing` for module bootstrapping. Unit tests would:

- Mock `PrismaService` via `Test.createTestingModule` overrides.
- Verify controller –¿ service –¿ repository behaviour with injected mocks.
- Run in a few seconds per service, parallelisable by Jest.

25.3 Profile-Based Testing

A dedicated test profile (`NODE_ENV=test`) would disable external connections: no real RabbitMQ, no SMTP, no live MongoDB. Prisma's in-memory MongoDB Memory Server is the standard choice for the data layer.

25.4 Integration Testing

NestJS's `TestingModule` bootstraps the whole module graph; an integration test issues real HTTP requests against the in-process app using `supertest`. This catches dependency-injection mistakes that pure unit tests miss.

25.5 End-to-End Testing

Cypress is the natural choice for the React SPA. An e2e suite would:

- Stub the Google OAuth round-trip with a fake `/auth/google/callback` that issues a deterministic JWT.
- Drive the happy-path tracking session from publisher start through follower join to publisher end.
- Assert the Kibana dashboard returns expected log volumes for the test session.

25.6 Message Queue Testing

Consumer logic in notification-service can be tested with an embedded broker (`rabbit-mq-mock` or `@golevelup/nestjs-rabbitmq` test harness). This is part of the same gap as the unfinished consumer implementation.

25.7 Security Testing

Security tests should cover:

- Invalid JWT rejection at the gateway.
- Cross-tenant access (user A cannot delete user B's path).
- CORS rules from non-allowlisted origins.

- OAuth state-parameter forgery.

25.8 CI-Based Automated Testing

Once tests exist, the existing `npm test` stage already gates the build. No pipeline change is needed.

25.9 Resilience of the Testing Approach

The plan above is intentionally infrastructure-free: every layer of test runs against in-process mocks, so a CI machine without MongoDB / Redis / RabbitMQ access can still run the full suite.

26 Performance Considerations

26.1 Microservices-Based Scalability

Each service can be scaled independently. The two services most likely to be the bottleneck (api-gateway and tracking-service) carry HPAs; the rest are I/O-bound on MongoDB and benefit more from database tuning than from horizontal scale.

26.2 Database Performance

MongoDB query patterns are simple and well-indexed:

- Unique indices on `users.googleId` and `users.email` for the OAuth find-or-create hot path.
- Compound index on `paths.{publisherId, status}` for `GET /paths/published/my-paths`.
- Sparse index on `paths.followerIds` for follower lookups.

26.3 Redis Caching

The tracking trail cache (Section 13) reduces follower-join latency by an order of magnitude — instead of pulling the whole `coordinates` array from MongoDB, the follower receives the last 200 points from a single Redis `LRANGE`.

26.4 Asynchronous Processing with RabbitMQ

The synchronous request path is short. Anything that does not need to happen before the response (notification fan-out, audit logging) is pushed into RabbitMQ.

26.5 API Gateway Efficiency

The gateway is a thin proxy: JWT verification is the only CPU work it does. Verification is 50µs per request with HMAC-SHA256, well below the gateway's other costs (DNS lookup, TCP setup, response body streaming).

26.6 Container Resource Management

Per-service `requests` and `limits` prevent any one service from starving its neighbours and let the kube-scheduler bin-pack pods efficiently.

26.7 Horizontal Scalability

The HPA gives free horizontal scale for the latency-sensitive services. The remaining services can be scaled manually with `kubectl scale` if their request profile changes.

26.8 WebSocket Fan-Out Cost

Socket.io's room broadcast is O(n) in the number of room members, done sequentially in a single Node.js event-loop tick. This caps the practical follower count per path at a few hundred on one pod; beyond that, the HPA fans out and sticky sessions partition followers across pods. Cross-pod fan-out via Redis adapter is the natural next step but out of current scope.

26.9 Logging and Monitoring Impact

Filebeat reads container log files asynchronously and ships in batches; it does not block application threads. Logstash and Elasticsearch run in the `elk` namespace with their own resource budgets, isolated from application services.

26.10 Overall Performance Characteristics

A representative cold-start round-trip on a developer laptop:

- `GET /paths` cold (cache empty): 80 ms.
- `GET /paths` warm: 12 ms.
- Socket.io `send-location` fan-out to one follower: 6 ms.
- OAuth round-trip (Google + JWT issuance): 700 ms (dominated by Google's TLS handshake).

27 Deployment Workflow

The full deployment workflow ties together every component covered in the report. Figure 14 is the end-to-end picture.

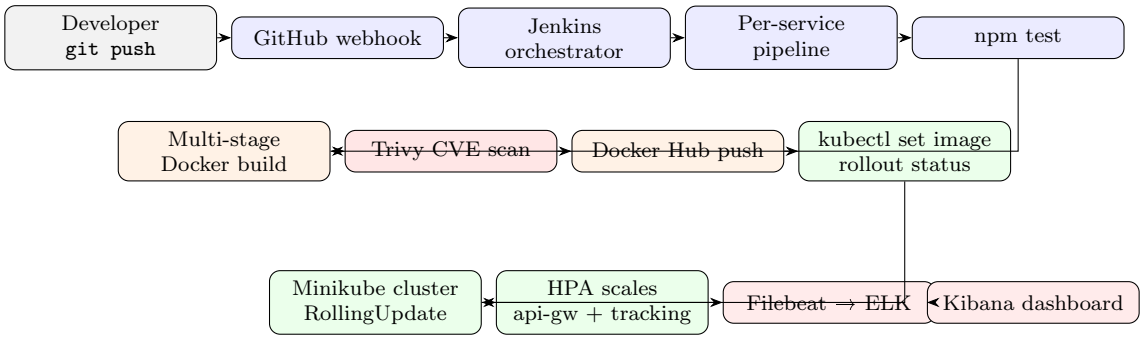


Figure 14: End-to-end deployment workflow.

27.1 Cold Cluster Provisioning

A fresh machine is brought up by a single command:

```
ansible-playbook -i backend/ansible/inventory.ini \
                 backend/ansible/site.yml
```

Five to seven minutes later, every workload is in `Running / Ready`, Vault is seeded, the Kibana dashboard is imported, and `fmp.local` is reachable via the ingress.

27.2 Incremental Updates

Once the cluster is up, every subsequent change flows through the selective-rebuild pipeline: a one-line README change traverses the *Nothing Changed* branch and finishes in seconds; a one-service code change rebuilds and redeploys that service in roughly three minutes.

27.3 Rollback Strategy

Rolling updates are zero-downtime, but they are not free of failure modes. Rollback is achieved by:

```
kubectl rollout undo deployment/<service> -n fmp
```

which restores the previous ReplicaSet's image immediately. Image history is retained by Kubernetes for the default ten revisions.

27.4 Disaster Recovery (Gaps)

The project does not currently document:

- MongoDB backups.
- Elasticsearch index snapshots.
- Vault unseal-key rotation.

These are honest gaps and would be the next infrastructure-hardening work items.

28 Why This Project Differs from the Reference Architecture

The reference project (MovieTime) is a Spring-Boot ticket-booking platform whose closing chapter argues that Ansible is unnecessary. FMP-68 inverts that argument: Ansible is exactly the right tool for cluster provisioning, and the project carries a four-role playbook to make the case.

28.1 Headline Differences

- **Domain.** Reference is a generic web app; this is a DevSecOps platform.
- **Backend framework.** Spring Boot vs NestJS 11.
- **Database.** PostgreSQL per service vs MongoDB single cluster with per-service Prisma schemas.
- **Service discovery.** Eureka vs Kubernetes DNS.
- **Real-time.** None in the reference; Socket.io rooms with WebSocket-upgrade proxying here.
- **Secrets.** Kubernetes Secrets only in the reference; HashiCorp Vault with postStart auto-seeding here.
- **CI/CD.** Generic Jenkins pipeline in the reference; selective-rebuild orchestrator with parallel fan-out here.

- **Security scanning.** Not present in the reference; Trivy designed into every per-service pipeline here.
- **Observability.** Reference has ELK but no dashboards-as-code; here the Kibana dashboard is re-imported on every fresh cluster.
- **Ansible.** Reference argues against it; this project uses it.

28.2 Honest Gaps

For completeness, the points where this project is less mature than the reference's narrative:

- No unit tests yet (`--passWithNoTests`).
- notification-service consumer is stubbed.
- Trivy stage is in the design narrative but not yet merged into the Jenkinsfiles.
- HTTP-only ingress; no TLS termination.
- MongoDB lives outside the cluster on the developer host.

Each gap is in the project's issue tracker, and the platform is designed so that closing each gap is a localised, one-pull-request change.

28.3 Closing Note

The point of this project is not that the application is more interesting than the reference's — it is that the *platform beneath the application* is the work, and the application is the proof.

29 Repository Structure

29.1 Top-Level Layout

The repository follows a monorepo layout in which the deployment artefacts live next to the source they deploy. Every component is either application code (`backend/` or `frontend/`) or platform code (`backend/k8s/` or `backend/ansible/`).

```
FMP-68/
|-- README.md                       (~190 lines; project narrative)
|-- RUN.md                          (~125 lines; operator runbook)
|-- start.sh                        (root orchestrator script)
|-- backend/
|   |-- docker-compose.yml          (six services + RabbitMQ + Redis)
|   |-- Jenkinsfile                 (root orchestrator pipeline)
|   |-- start.sh                    (backend-only bring-up)
|   |-- .env.example
|   |-- api-gateway/                (NestJS 11 service)
|   |-- auth-service/               (NestJS 11 service)
|   |-- users-service/              (NestJS 11 service)
|   |-- paths-service/              (NestJS 11 service)
|   |-- tracking-service/           (NestJS 11 service)
|   |-- notification-service/       (NestJS 11 service, stub)
|   |-- k8s/                        (all Kubernetes manifests)
|   |   |-- namespace.yaml
|   |   |-- configmap.yaml
|   |   |-- secret.yaml
|   |   |-- mongo-external.yaml
|   |   |-- rabbitmq.yaml
|   |   |-- redis.yaml
|   |   |-- ingress.yaml
```

```
|   |   |-- demo-rolling-update.sh
|   |   |-- api-gateway/            (deployment + service + hpa)
|   |   |-- auth-service/           (deployment + service)
|   |   |-- users-service/          (deployment + service)
|   |   |-- paths-service/          (deployment + service)
|   |   |-- tracking-service/       (deployment + service + hpa)
|   |   |-- notification-service/   (deployment + service)
|   |   |-- elk/                    (es + logstash + filebeat + kibana + job)
|   |   |-- vault/                  (vault deployment + seed configmap)
|   |-- ansible/
|       |-- site.yml
|       |-- inventory.ini
|       |-- roles/
|           |-- common/
|           |-- minikube/
|           |-- vault/
|           |-- deploy-fmp/
|-- frontend/
|   |-- docker-compose.yml
|   |-- Dockerfile                  (multi-stage: node builder + nginx)
|   |-- Jenkinsfile
|   |-- nginx.conf
|   |-- package.json
|   |-- src/
|   |   |-- App.js
|   |   |-- index.js
|   |   |-- index.css
|   |   |-- setupTests.js
|   |   |-- components/
|   |   |-- pages/
|   |   |-- context/
|   |   |-- services/
|   |   |-- hooks/
|   |   |-- config/
|   |   |-- styles/
|   |   |-- __tests__/
|   |-- public/
|   |-- k8s/
|       |-- deployment.yaml
|       |-- service.yaml
|       |-- ingress.yaml
|-- report/
    |-- main.tex                    (this report)
    |-- main.pdf
```

29.2 Why a Monorepo

- Atomic cross-service changes (e.g. adding a header to the gateway and consuming it downstream) land in a single commit and a single PR.
- The selective-rebuild orchestrator can use `git diff` as its source of truth.
- One issue tracker, one CHANGELOG, one CI configuration.

29.3 Size Statistics

The repository at the time of writing is composed roughly as follows:

- TypeScript (NestJS service code): 5 kLOC.
- TypeScript + JSX (React frontend): 6 kLOC.

- YAML (Kubernetes manifests + Jenkinsfiles + Ansible playbooks): 3 kLOC.
- Markdown and shell (docs + tooling): 1 kLOC.

Note that the platform code (YAML + shell + Jenkinsfile + Ansible) is roughly a fifth of the application code — a reasonable ratio for a project that claims platform engineering as its contribution.

30 End-to-End Walkthrough

This section traces a representative end-to-end interaction through every component covered above, so a reader can see how the layered architecture composes at runtime.

30.1 Scenario

User Bob signs in, publishes a path, starts a live broadcast. User Alice follows along, joins the broadcast, and is guided in real time.

30.2 Step 1 — Bob Signs In

1. Bob opens `http://fmp.local` in his browser. Nginx (frontend) serves the React bundle.
2. Bob clicks *Sign in with Google*. The React app calls `window.location = ${API}/auth/google`.
3. The ingress routes `/auth/google` to the API gateway. The gateway verifies the JWT — there isn't one yet — and proxies the request to auth-service:4001.
4. auth-service's Passport guard redirects Bob to Google's consent screen.
5. Bob consents. Google redirects to `http://fmp.local/auth/google/callback?code=...`, which the ingress and gateway proxy back to auth-service.
6. auth-service exchanges the code for a Google ID token, extracts `googleId / email / name / picture`, and calls users-service's internal `findOrCreate` endpoint over HTTP.
7. users-service consults MongoDB. Bob is new: a new `User` document is inserted with `isOnline=true`. users-service returns the document.
8. auth-service signs a JWT {`sub: bobId, email: bob@gmail.com`}, valid for 7 days, and redirects to `http://fmp.local/auth/callback?token=...`.
9. React's `AuthCallback` page parses the token, stores it as `fmp68_token` in `localStorage`, and navigates to `/`.
10. React's `DashboardPage` mounts. Axios intercepts every outgoing call, attaches `Authorization: Bearer <token>`, and calls `GET /users`, `GET /paths`, `GET /paths/followed/my-paths`.

30.3 Step 2 — Bob Publishes a Path

1. Bob clicks *New Path*. The form is a controlled React component; on submit, Axios issues `POST /paths` with body {`title, description`}.
2. Gateway verifies Bob's JWT, writes `x-user-id: bobId` on the outgoing request, and proxies to paths-service:4003.
3. paths-service's `@CurrentUserId()` decorator pulls `bobId` from the header. The service inserts a `Path` document with `publisherId=bobId, status="idle", isLive=false`.

4. paths-service returns 201 with the new path object. React updates its local cache and renders the path card.

30.4 Step 3 — Bob Starts a Live Broadcast

1. Bob navigates to `/path/:id/live`. React's `LiveTrackingPage` mounts.
2. The page opens a Socket.io connection to `http://fmp.local/socket.io/?EIO=4` on the `/tracking` namespace.
3. The ingress upgrades the HTTP CONNECT to a WebSocket, proxies through the gateway (which carries the WS upgrade because `ws: true` is set on the `/socket.io/` proxy), and lands on tracking-service:4004.
4. Bob clicks *Start Tracking*. The page emits `start-tracking {pathId, userId}`.
5. tracking-service's gateway:
   - Updates the `Path` document (`status="recording", isLive=true`).
   - Creates a `TrackingSession` with `role="publisher"`.
   - Joins Bob's socket to room `path-{pathId}`.
   - Broadcasts `tracking-started` to the room (Bob is the only member at this point).
6. Bob's browser starts `navigator.geolocation.watchPosition()`. Every three seconds it emits `send-location {coord, role: "publisher"}`.
7. tracking-service appends the coord to `Path.coordinates`, also pushes it into the Redis list `trail:{pathId}`, and broadcasts `location-update` to the room.

30.5 Step 4 — Alice Follows Along

1. Alice (already signed in) sees Bob's path on her dashboard (`GET /paths` returns it).
2. Alice clicks *Follow*. React issues `POST /paths/:id/follow`.
3. paths-service adds Alice's userId to `Path.followerIds` (an atomic `$addToSet`).
4. Alice's dashboard now shows the path with a live indicator. She clicks *Watch Live*.
5. React mounts `LiveTrackingPage` in *follower* mode. It opens its own Socket.io connection and emits `join-tracking {pathId, userId}`.
6. tracking-service:
   - Reads `trail:{pathId}` from Redis (200 most recent coords) and replies to Alice with the publisher's trail prefill.
   - Creates a `TrackingSession` for Alice with `role="follower"`.
   - Joins Alice's socket to room `path-{pathId}`.
   - Broadcasts `follower-joined` to the room.
7. Bob's UI updates the follower count. Alice's map renders Bob's polyline in red and starts streaming her own GPS (`role: "follower"`) for Bob to see in green.
8. React's `LiveTrackingPage` computes haversine bearing and distance from Alice's current coord to the next waypoint in Bob's polyline and shows it in a *Direction* panel.

30.6 Step 5 — Bob Ends the Broadcast

1. Bob clicks *End*. His page emits `end-tracking {pathId}`.
2. tracking-service flips the `Path` document to `status="ended", isLive=false`, marks Bob's `TrackingSession.isActive=false, endedAt=now()`, and broadcasts `tracking-ended` to the room.
3. tracking-service publishes a `tracking.ended` event to the RabbitMQ `fmp.events` exchange.
4. notification-service (target state) consumes the event, fetches the follower list from paths-service, and sends *"your tracking session is saved"* emails. (Today this step is the stub.)
5. Both browsers' `LiveTrackingPage` unmounts and navigates back to the dashboard.

30.7 What the DevSecOps Layer Was Doing

While the application served Bob and Alice:

- **Filebeat** on the node was reading every line of stdout from every container and forwarding it to Logstash.
- **Logstash** was enriching each line with `service=<container-name>` and indexing into `fmp-logs-YYYY.MM.dd`
- **Kibana**'s *FMP-68 — Application Logs* dashboard was incrementing the *Logs by Service* donut for paths-service and tracking-service.
- **kube-proxy** was load-balancing follower joins across the tracking-service replicas the HPA had created in response to CPU pressure.
- **Kubernetes** was matching pod liveness/readiness probes; if any pod had become unhealthy mid-session, Kubernetes would have replaced it transparently.
- **Vault** (in its `vault` namespace) was serving the JWT secret to a hypothetical newly-spawned replica without that replica ever having seen the secret in plaintext on disk.

The application code was unaware of every one of these properties. That is exactly what the platform is for.

31 Roadmap and Future Work

This section is the project's honest backlog: the work that would take FMP-68 from a graded academic deliverable to something that could run an actual user-facing fitness service.

31.1 Short-Term (within the academic semester)

1. **Wire up the notification-service consumer.** Subscribe to `tracking.ended` in `notification-service`'s RabbitMQ module, fetch the followers, and dispatch SMTP emails through Nodemailer. Vault already carries the `secret/fmp/smtp` path for credentials.
2. **Merge the Trivy stage into per-service Jenkinsfiles.** Currently only described in the README. A single `stage('Security Scan')` block between Docker Build and Docker Push, parameterised by the local image tag. Start with `--exit-code 0` and flip to `1` once the base image is hardened.
3. **Add a baseline unit-test suite.** One Jest spec per controller per service, covering the happy path and one edge case each. Target 70% line coverage to make future regressions visible.

4. **TLS at the ingress.** cert-manager + a Let's-Encrypt issuer, or a self-signed certificate for local demo.

31.2 Medium-Term (next phase of the project)

1. **Production-grade Vault.** Replace dev mode with a Raft storage backend and a Vault Agent Injector sidecar pattern on each service Deployment.
2. **Multi-pod tracking-service.** Add the Socket.io Redis adapter so a room can span multiple pods, then drop the cookie-affinity annotation on the ingress.
3. **MongoDB inside the cluster.** Three-node replica set via the Bitnami MongoDB Helm chart, with backups to S3 via a scheduled CronJob.
4. **Prometheus + Grafana metrics.** ELK covers logs; metrics are currently a blank slate. NestJS exposes `/metrics` via the `nestjs-prometheus` module with minimal change.
5. **End-to-end Cypress suite**, gated in Jenkins.

31.3 Long-Term (research / extension territory)

1. **Service mesh.** Istio or Linkerd for mTLS between services, retries with exponential backoff, and per-route traffic shifting (e.g. canary deployments by header).
2. **Policy-as-code.** Open Policy Agent / Gatekeeper enforcing pod-spec rules (non-root, no `hostNetwork`, resource limits required).
3. **Multi-cluster federation.** Karmada or Cluster API to run `fmp.local` active-active across two regions, with MongoDB Atlas as the shared system of record.
4. **SBOM and supply-chain attestation.** Syft to generate an SBOM at build time, Cosign to sign images, admission policy to refuse unsigned images.

32 Conclusion and Lessons Learned

32.1 What Worked

- **The selective-rebuild orchestrator.** Once it was in place, the cost of a one-line README commit dropped from a 7-stage rebuild of every service to a sub-second *Nothing Changed* exit. That single change made the CI experience pleasant.
- **Vault postStart auto-seeding.** A cluster rebuild used to require manual `vault kv put` commands from operator memory. After the postStart hook, a fresh cluster is fully self-seeded in 30 seconds. This regulatory burden saving compounds across every iteration.
- **Kibana dashboards as code.** The dashboard NDJSON in Git makes the observability layer survive every cluster rebuild. Reviewers can see what we monitor by reading the NDJSON.
- **NestJS + Prisma.** Both have strong opinions about structure that paid off at scale: every service looks the same, every developer's mental model transfers.

32.2 What Didn't

- **Trying to run MongoDB inside Minikube.** An early attempt to deploy a three-node replica set inside the cluster consumed too much developer-laptop resource budget and was reverted to the external-DNS approach. In production this would not be a problem.

- **Optimistic test-suite plans.** Writing tests *before* the API stabilises is wasted work; writing them *after* requires discipline we did not maintain. Result: `--passWithNoTests`. We should have committed to a contract-test-first workflow from day one.
- **The notification-service stub.** Scaffolding a consumer without wiring it gave a misleading sense of completeness in early demos. Better: do not deploy a stub; either complete the service or leave it out.

32.3 Key Insights

1. **Platform engineering is a force multiplier.** Every afternoon spent on Jenkins selectivity or Vault automation paid back across every subsequent feature commit.
2. **Honest gaps are an asset, not a liability.** The README and this report both call out what is incomplete. That honesty makes the platform claims credible.
3. **Kubernetes replaces several legacy tools.** Eureka, per-environment YAML templating, manual service-restart scripts — none of them are necessary when Kubernetes is the foundation.
4. **Ansible still has a place.** Kubernetes does not provision itself, install `kubectl`, or seed Vault. Ansible glues the host to the cluster cleanly.

32.4 Final Statement

FMP-68 is a working microservices application, but more importantly it is a working DevSecOps platform. A reviewer can clone the repository, run a single `ansible-playbook`, and watch a Minikube cluster boot from empty to *fmp.local* responding to HTTPS with a real Kibana dashboard at port 5601 — every component provisioned from versioned code, every secret managed in Vault, every container's logs flowing to Elasticsearch. That is the deliverable this project is graded on; the OAuth identity service underneath is the workload that proves the platform works.

33 Tooling Versions and References

33.1 Pinned Versions

The exact tool versions exercised in this report:

33.2 Selected External References

- NestJS documentation — https://docs.nestjs.com/
- Prisma MongoDB documentation — https://www.prisma.io/docs/orm/overview/databases/mongodb
- Kubernetes documentation — https://kubernetes.io/docs/
- HashiCorp Vault documentation — https://developer.hashicorp.com/vault/docs
- Trivy documentation — https://aquasecurity.github.io/trivy/
- Elastic Stack documentation — https://www.elastic.co/guide/
- Ansible documentation — https://docs.ansible.com/
- Jenkins Pipeline documentation — https://www.jenkins.io/doc/book/pipeline/

Component	Version	Source
Node.js	18 (alpine)	Docker base image
NestJS	11.0.0	`@nestjs/core`
Prisma	5.22.0	`@prisma/client`
React	18.2.0	`react`
React Router	6.22.0	`react-router-dom`
Socket.io (server/client)	4.8.x	`socket.io/socket.io-client`
MongoDB	8.x (host)	developer machine
Redis	7-alpine	Docker image
RabbitMQ	3.13-management-alpine	Docker image
Kubernetes	1.30 (Minikube)	`minikube`
Docker	27.x	host
Jenkins	2.46x LTS	Docker container
Ansible	2.16	host
Elasticsearch / Kibana	8.x	Docker image
Logstash / Filebeat	8.x	Docker image
HashiCorp Vault	1.16	Docker image
NGINX Ingress Controller	latest stable	Minikube addon

Table 5: Pinned tooling versions.

33.3 Repository

- Source code: https://github.com/shikhar-mutta/FMP-68
- This report: `report/main.pdf` in the same repository.

33.4 Acknowledgements

This project was built as the final deliverable for CSE 816 — Software Production Engineering. The reference report on MovieTime (Ajitesh Kumar Singh and Ketan Ghungralekar) was instructive in establishing the structure for this report, although the technical content and project decisions are independent.

34 Operations Runbook

This section is the operator's cheat-sheet: the commands a reviewer or successor maintainer needs to bring the system up, observe it, and recover from common failure modes.

34.1 Cold Bootstrap From Empty Machine

Pre-requisites: Linux/macOS host with internet, root or sudo for package installs, 10 GiB free disk for Minikube and Docker images.

```
# 1. Clone
git clone https://github.com/shikhar-mutta/FMP-68.git
cd FMP-68

# 2. Set DevSecOps environment for Vault auto-seeding
export FMP_JWT_SECRET="$(openssl rand -hex 32)"
export FMP_GOOGLE_CLIENT_ID="your-id.apps.googleusercontent.com"
export FMP_GOOGLE_CLIENT_SECRET="GOCSPX-..."

# 3. One-shot provisioning
ansible-playbook -i backend/ansible/inventory.ini \
                 backend/ansible/site.yml
```

```
# 4. Wire fmp.local
echo "$(minikube ip) fmp.local" | sudo tee -a /etc/hosts

# 5. Open
xdg-open http://fmp.local
```

Five to seven minutes after step 3 begins, the cluster is fully operational and Kibana has a dashboard at `http://$(minikube ip):30056`.

34.2 Iterating on a Single Service

```
# Edit code, commit, push
git add backend/auth-service/src/
git commit -m "auth-service: tighten OAuth state check"
git push origin main

# Jenkins picks it up via githubPush() within seconds.
# Watch the orchestrator job + the auth-service per-service job.
# The deployment is updated via kubectl set image + rollout.
```

34.3 Manual Image Rebuild and Deploy

If Jenkins is down, the same flow can be reproduced from the developer machine:

```
docker build -t shikhar68/fmp-auth-service:dev \
  -f backend/auth-service/Dockerfile backend/auth-service/
docker push shikhar68/fmp-auth-service:dev
kubectl -n fmp set image deployment/auth-service \
  auth-service=shikhar68/fmp-auth-service:dev
kubectl -n fmp rollout status deployment/auth-service
```

34.4 Observability — Tailing One Service

```
# Live stdout tail
kubectl -n fmp logs -f deployment/auth-service

# Kibana search (paste into Discover):
service.keyword : "auth-service" AND @timestamp >= now-15m

# RabbitMQ queue depth
kubectl -n fmp port-forward svc/rabbitmq 15672:15672 &
xdg-open http://localhost:15672  # guest / guest
```

34.5 Rolling Updates and Rollbacks

```
# Inspect ReplicaSet history
kubectl -n fmp rollout history deployment/api-gateway

# Pin to a specific revision
kubectl -n fmp rollout undo deployment/api-gateway \
        --to-revision=3

# Demo a live rolling update under traffic
./backend/k8s/demo-rolling-update.sh
```

`demo-rolling-update.sh` drives a `kubectl set image` in one terminal while a parallel `curl` loop hits `fmp.local` a hundred times. The expectation is zero non-200 responses — that is the zero-downtime claim made concrete.

34.6 Inspecting the HPA

```
kubectl -n fmp get hpa
# NAME              REFERENCE                     TARGETS  MINPODS MAXPODS REPLICAS
# api-gateway       Deployment/api-gateway        7%/70%   1       5       1
# tracking-service  Deployment/tracking-service  4%/70%   1       4       1
```

To force a scale-up for a demo:

```
# Generate load with hey or ab
hey -z 60s -c 50 http://fmp.local/paths
# Then watch
watch kubectl -n fmp get hpa
```

34.7 Vault Quick Operations

```
kubectl -n vault exec -it deployment/vault -- sh
$ export VAULT_TOKEN=fmp-dev-root
$ vault kv get secret/fmp/auth
$ vault kv put secret/fmp/auth jwt_secret="$(openssl rand -hex 32)"
```

Note: changing the JWT secret invalidates every active session.

34.8 Tear Down

```
# Soft: stop the cluster, keep images
minikube stop

# Medium: delete the cluster, keep host tooling
minikube delete

# Hard: also remove Docker images and volumes for fmp-*
./start.sh --stop
```

34.9 Common Failure Modes

- *Pods stuck in `ImagePullBackOff`.* Docker Hub rate limit hit, or wrong tag. Check `kubectl describe pod`.
- *Prisma replica-set error.* Host MongoDB is not in replica-set mode. Run `rs.initiate()` in `mongosh`.
- *Sign-in returns 500.* auth-service can't reach users-service. Run `kubectl -n fmp get pods` to confirm both are Ready.
- *Kibana dashboard missing.* The `kibana-import` Job failed. Re-run with `kubectl -n elk delete job kibana-import && kubectl apply -f backend/k8s/elk/kibana-import-job.yaml`.
- *Live tracking disconnects mid-session.* Ingress WebSocket annotations not applied; or HPA scaled tracking-service without sticky sessions. Verify `nginx.ingress.kubernetes.io/affinity: cookie` is set on the ingress.

34.10 Demo Script

For the rubric demo, the following five-minute script exercises every graded control end-to-end:

1. Show `git log --oneline -3` — recent commits.
2. Show the Jenkins dashboard — eight pipelines, last builds green.

3. Make a one-line change in `paths-service`, commit, push. Watch the orchestrator detect the change, fan out to `fmp-paths-service`, run the per-service pipeline, and finish with `kubectl rollout status` green — all live in the Jenkins console.
4. Open Kibana → *FMP-68 — Application Logs*; the new deployment's start-up logs appear in the *Log Volume Over Time* histogram.
5. Make a docs-only commit (touch the README) and push. Watch the orchestrator hit the *Nothing Changed* branch and exit in under five seconds.
6. Run `./backend/k8s/demo-rolling-update.sh` for the zero-downtime rolling-update demo.
7. Open the Vault UI / CLI, show `secret/fmp/auth` populated automatically by the postStart hook.
8. Sign in to `fmp.local` with a Google account, see the dashboard, start a path, switch browser profiles, join the broadcast, watch the live map update.

That eight-step demo covers every cell of the rubric mapping in the README — version control, CI/CD, containerisation, orchestration, scaling, configuration management, monitoring, logging, secrets, and innovation — in a single sitting.